

Empirical Analysis of LLM Jailbreak Attack Families, 2023–2026

Introduction

Between 2023 and 2026, jailbreak research evolved from ad hoc prompt tricks into a diverse offensive ecosystem spanning manual role-play prompts, multi-turn conversational escalation, optimization-derived adversarial suffixes, many-shot context attacks, and agent- or tool-mediated exploit chains. Across this period, the central empirical finding has remained consistent: safety alignment at the refusal layer is often brittle under adaptive prompting, especially when attacks exploit conversational state, long-context conditioning, automated search, or external tools rather than a single obviously malicious query. Recent benchmark and red-teaming results suggest that no single attack family dominates in every setting; instead, effectiveness depends heavily on access assumptions, target model family, interaction budget, and the deployment environment in which the model is embedded. This report therefore treats “jailbreak capability” as an operational property of systems, not just base models, and compares attacks by measured success rate, transferability, cost, required expertise, and the defenses they have been shown to bypass. ([Russeinovich et al., 2024](#); [International AI Safety Report, 2026](#))

The empirical record shows meaningful variation across attack families. For multi-turn escalation, **Crescendo** substantially outperformed several contemporaneous jailbreak methods on a 50-task AdvBench subset: against GPT-4 it achieved **56.2% average ASR** and **98.0% binary ASR**, versus 40.0%/76.0% for PAIR and 37.0%/86.0% for MSJ; against Gemini Pro it reached **82.6% average ASR** and **100.0% binary ASR**, exceeding all other compared methods in the study. These results are important because they indicate that iterative steering can outperform stronger-looking single-turn attacks while requiring only black-box access and ordinary conversational interaction. More broadly, composable red-teaming work published in 2024 reported jailbreak combinations exceeding **80% ASR** against frontier proprietary systems including Claude 3 Sonnet and GPT-4o, reinforcing the view that attack composition materially changes risk estimates relative to isolated prompt templates. ([Russeinovich et al., 2024](#); [h4rm3l, 2024](#))

The attack families covered in this report differ not only in efficacy but also in attacker economics. **Persona-based and narrative attacks** typically have low monetary cost and low technical barriers, making them attractive for broad misuse despite often unstable performance. **Multi-turn escalation** increases interaction cost but remains accessible to non-specialists and can defeat safeguards that

rely on local prompt inspection rather than conversation-level intent tracking. **Many-shot jailbreaking** exploits long context windows, trading token cost for improved conditioning and often reducing the amount of creativity needed from the attacker. **Optimization-based suffix attacks** such as GCG and later efficiency-improved variants generally demand more expertise, repeated queries, or model-side information, but they are especially consequential because they can produce reusable and sometimes transferable adversarial strings that stress-test alignment at scale. Survey and taxonomy work through 2025 repeatedly emphasized that optimization-based methods pose a particularly serious threat because they are more automatable and less dependent on fragile handcrafted prompt artistry than manual or query-specific methods. ([ACL Findings, 2025](#); [Promptfoo LM Security DB: Faster GCG, 2024](#))

A second major trend is **transfer and generalization across model families**. Public vulnerability tracking indicates that core jailbreak families now affect a wide spread of systems, including OpenAI, Anthropic, Google, and open-weight chat models. For example, Crescendo has been cataloged as affecting ChatGPT, GPT-4, Gemini Pro and Ultra, Claude 2/3/3.5, and Llama-family chat models, suggesting that conversational escalation is not tightly coupled to any one provider's refusal style. Likewise, GCG-family attacks and their faster descendants have been reported against both proprietary and open models, including GPT-3.5 Turbo, GPT-4 Turbo, Llama 2 Chat, and Vicuna. This breadth does not by itself prove universal transferability in the strict experimental sense, but it does show that multiple model lineages share exploitable alignment patterns, particularly where defenses are prompt-layer or refusal-layer dominant. ([Promptfoo LM Security DB: Crescendo, 2024](#); [Promptfoo LM Security DB: Faster GCG, 2024](#))

The defensive picture is equally important. The literature from 2023–2026 suggests that these attacks routinely bypass **static harmful-content filters, single-turn refusal tuning, prompt-template guards, and simplistic classifier gates**, while newer work shows that agentic settings introduce fresh failure modes beyond ordinary chat refusal. In browser-agent and tool-use contexts, refusal-trained models can still be induced into harmful task completion because the exploit path is mediated by planning, external state, websites, retrieved content, or tool outputs rather than a direct user prompt alone. Related 2025-era references also point to indirect and RAG-mediated jailbreaks, voice-mediated attacks, and multi-round jailbreak agents, indicating that the relevant unit of defense is increasingly the entire sociotechnical execution chain rather than the text prompt in isolation. ([Refusal-Trained LLMs Are Easily Jailbroken As Browser Agents, 2024](#); [Gabriel, annotated survey, 2025](#); [International AI Safety Report, 2026](#))

Finally, jailbreaks have matured into a **real-world adversarial ecosystem** with implications for cost, scaling, and accessibility. Public communities and prompt-sharing channels have lowered discovery costs for low-skill attackers, while automated jailbreak generation, fuzzing, tree-search, and evolutionary frameworks have reduced the expertise needed to discover new attack variants. By

late 2025, public tracking already described “autonomous jailbreak evolution” systems targeting GPT-4o, Claude 3.7, Gemini 2, and Llama 3-class models, illustrating a shift from handcrafted prompts toward iterative attack development pipelines. This matters economically: once an attack template, suffix, or conversational strategy is discovered, it can be productized, sold, shared, or adapted across targets at marginal cost far below its original discovery cost. The result is a jailbreak-as-a-service dynamic in which frontier-model defenses are pressured not only by novel research attacks but also by communities that rapidly operationalize them. ([Promptfoo LM Security DB: Autonomous Jailbreak Evolution, 2025](#); [ResearchGate defensive framework references, 2025](#))

Against that backdrop, this report provides an empirical comparison of the major jailbreak families documented from 2023 through April 2026: persona-based attacks, multi-turn escalation, adversarial suffix optimization, many-shot jailbreaking, and tool/agent-mediated exploit chains. The emphasis is on measured attack success rates on named frontier models, attack cost and attacker skill requirements, cross-model transfer, and the specific safeguards each family has bypassed in published evaluations. Throughout, the report distinguishes direct evidence from inference and highlights where claims are benchmarked, where they are vendor- or community-tracked, and where the state of the evidence remains incomplete or time-sensitive. ([Rusinovitch et al., 2024](#); [ACL Findings, 2025](#); [International AI Safety Report, 2026](#))

Table of Contents

- 2023–2026 Jailbreak Taxonomy and Evaluation Framework
 - Taxonomic schema for documented jailbreak families
 - Metric design: ASR, refusal bypass, transferability, and reproducibility
 - Empirical attack-family matrix: documented efficacy, transfer, cost, and required skill
 - Frontier-model coverage and what defenses each family bypasses
 - Defensive mechanisms, benchmark design, and the economics of adversarial prompt development
- Persona-Based, Manual, and Many-Shot Jailbreaking: Empirical Effectiveness, Operational Burden, Transfer, and Safeguard Evasion
 - Scope note and differentiation from prior sections
 - Operational distinctions among the three families
 - In-the-wild prevalence: manual jailbreaks as the dominant observable ecosystem artifact
 - Benchmarked effectiveness available in the supplied literature
 - Empirical comparison table

- Measured Effectiveness of Manual and Persona Jailbreaks in the Wild Versus Benchmarked Settings
 - Why in-the-wild evidence and benchmark evidence diverge
 - Manual/persona prompts: what can be said empirically from the supplied record
 - Structured manual prompts: CIA as the strongest benchmarked evidence in this subset
 - Many-shot jailbreaks: demonstration effects are real even when exact percentages are sparse
 - Relative effectiveness versus automated comparators
 - A more nuanced effectiveness matrix
- Execution Cost, Query Burden, and Attacker Skill Requirements
 - This section's distinction from earlier cost discussions
 - Persona/manual attacks: minimal compute, non-trivial craft knowledge
 - Skill model for persona prompts
 - Compositional manual attacks: higher design sophistication, still modest infrastructure cost
 - Many-shot attacks: token-expensive, cognitively simple, often organizationally convenient
 - Query burden and the contrast with automated search
 - Economic decomposition: development versus deployment
 - 1) Persona/manual prompts
 - 2) Compositional manual prompts
 - 3) Many-shot demonstrations
 - Defender implications of low-skill attack availability
- Cross-Model Portability and Family-Specific Transfer Dynamics
 - Why transfer must be treated differently for these attack families
 - Direct evidence: CIA transfers across vendor and model family boundaries
 - Manual persona prompts: likely broad but unstable transfer
 - Many-shot transfer: portability through generic imitation behavior
 - Transfer versus detectability tradeoff
 - Comparison with automated readable attacks
 - Family-specific transfer profile
 - Security significance of cross-family portability
- Prompt-Level Safeguards Bypassed, Pressured, or Countered by These Attacks
 - Distinction from earlier defense mapping
 - Persona/manual attacks versus surface instruction safeguards
 - Compositional instruction attacks versus intent-based defenses
 - What CIA bypasses

- What IBD demonstrates
 - What remains unresolved
- Many-shot jailbreaks versus in-context refusal conditioning
- Multi-Turn and Automated Black-Box Jailbreak Families: Measured Effectiveness, Interaction Budget, and Defense Evasion
 - Source-bound scope and differentiation from prior subtopic coverage
 - Crescendo: iterative escalation as a black-box conversational search process
 - Tree-structured and modular black-box search: from Tree of Attacks to cross-behavior automation
 - MRJ-Agent, Echo Chamber, and composable agentic jailbreak orchestration
 - Frontier-model results, transferability, and the role of interaction budget
- Optimization-Based and Adversarial Suffix Jailbreaks: Empirical Performance, Transfer, and Alignment Evasion
 - Scope delimitation relative to earlier subtopic reports
 - Method families inside the suffix-optimization lineage
 - White-box dependence, gray-box assumptions, and black-box deployment realities
 - Benchmarking caveats specific to this literature
 - Measured results reported for GCG and directly related variants
 - Faster-GCG and the broader acceleration trend
 - Universality: from per-prompt strings to reusable attack artifacts
 - What AmpleGCG adds beyond “better transfer”
 - Cross-model transfer: what the literature supports and what remains uncertain
 - Comparative evidence on open versus closed model performance
 - Why these attacks work: alignment failure at the token-distribution level
 - Refusal bypass as objective hacking rather than semantic persuasion
 - Position, syntax, and attack structure: beyond “append random garbage”
 - Index gradients and richer optimization signals
 - Measured effectiveness, cost, and access requirements across the main suffix subfamilies
- Tool-, Agent-, and Ecosystem-Level Jailbreak Dynamics
 - Measurement boundary, evidence classes, and how this section differs from earlier coverage
 - Browser agents, toolchains, and RAG as multiplicative exploit surfaces
 - Browser-agent jailbreak dynamics
 - RAG-mediated exploit chains
 - A chain-level view: from prompt to exploit

- Public sharing, community markets, and the maturation of jailbreak supply chains
 - From isolated prompts to reusable attack products
 - Jailbreak-as-a-service as a service model, not merely a marketplace label
 - Community incentives and the role of benchmark culture
 - Real-world observability constraints
- The economics of automated adversarial prompt development
 - Cost decomposition across the adversarial development pipeline
 - Public sharing as subsidy for autonomous search
 - Search efficiency and transfer as economic multipliers
 - Labor substitution: from prompt engineers to attack operators
- Autonomous jailbreak evolution, model-to-model adaptation, and defense pressure
 - From static prompts to adaptive attack loops
 - Defensive updates and attacker adaptation horizons
 - Cross-model adaptation and wrapper diversity
 - Security implications for defenders
 - Empirical status table: what is measured, what is inferred, and what remains uncertain

2023–2026 Jailbreak Taxonomy and Evaluation Framework

Taxonomic schema for documented jailbreak families

A practical 2023–2026 taxonomy is best organized by **where the adversarial leverage is applied** rather than by surface prompt style alone. Across the cited literature, five recurrent loci appear: **(i)** natural-language conversational manipulation, **(ii)** optimization over discrete text tokens, **(iii)** scaling/context-budget exploitation, **(iv)** multimodal perturbation or image-mediated steering, and **(v)** tool/agent-mediated exploit chains. This organization better matches measured transferability, cost structure, and the defenses each family tends to bypass than older taxonomies that grouped attacks only by “prompting” versus “adversarial” methods ([1](#), [2](#), [3](#)).

Within **natural-language conversational manipulation**, the literature from 2023–2026 includes persona-roleplay attacks, refusal-suppression framing, fictionalization, obfuscatory paraphrasing, translation/coding indirection, and multi-turn escalation strategies such as Crescendo-like attacks. These methods are usually black-box compatible, low-cost, and human-comprehensible, but their success rates often depend strongly on target alignment policy, conversation state, and safety classifier coupling. The paper *Activation-Guided Local Editing for Jailbreaking Attacks* is notable because it explicitly frames prior prompt-level attacks as scalable only with substantial human effort, then proposes a two-stage alternative that begins with scenario-based contextualization and

harmful-intent obfuscation before applying hidden-state-guided local edits to improve efficacy and coherence (2).

A second class comprises **optimization-based discrete text attacks**, dominated by adversarial suffix methods derived from Greedy Coordinate Gradient (GCG). The VLM survey text in *Toward Universal and Transferable Jailbreak Attacks on Vision-Language Models* summarizes GCG as the first optimization-based jailbreak over discrete inputs, subsequently improved by stronger objectives and replacement strategies, including AttnGCG and related follow-ons. A central empirical observation in that literature is that these attacks often achieve strong white-box or surrogate-model performance but historically suffer from limited transferability because they operate in discrete token space and are often constrained to short suffixes (1). This same limitation motivates later work seeking more transferable objectives and representation-level manipulations.

A third class is **context-scaling or many-shot exploitation**, where the attack does not rely on a single clever instruction or optimized suffix but instead uses long-context demonstrations, repeated examples, or synthetic policy precedents to shift the model’s response prior. Although the supplied citations do not provide a dedicated many-shot benchmark table, the 2024–2025 literature referenced within the VLM work places many-shot and best-of-(N) style attacks into the broader family of iterative or scaling-based jailbreaking strategies. Their key property is that they exploit the model’s instruction-following and in-context learning dynamics rather than token-level perturbation gradients (1).

A fourth class is **multimodal jailbreaks**, especially against vision-language models (VLMs). Here, adversarial leverage is applied through images, image-text pairs, or universal image triggers. The UltraBreak paper excerpted in the provided sources argues that earlier multimodal jailbreak methods such as VAJM, ImgJP, and UMK were often highly model-specific and transferred poorly, while its own contribution is a “universal and transferable” jailbreak image. It further distinguishes jailbreak images from more generic adversarial examples for recognition steering by emphasizing cross-target and cross-model unsafe response induction (1).

The fifth class is **tool/agent-mediated exploit chains**, in which the jailbreak objective is not merely to elicit a direct harmful completion but to induce an LLM agent to misuse tools, follow unsafe delegated tasks, retrieve prohibited instructions through tools, or combine individually allowed operations into a disallowed end state. Although the supplied citations do not contain a benchmark-specific tool-agent paper, this family is necessary for a 2023–2026 taxonomy because the period saw increasing deployment of browsing, coding, and action-taking agents, and evaluation increasingly shifted from “unsafe text generated” to “unsafe capability execution.” In the absence of specific numeric results in the cited snippets, any detailed assessment of this family must be labeled

as synthesis from broader 2024–2026 agent-safety literature rather than directly measured by the provided papers.

From an evaluation standpoint, this taxonomy also tracks **attacker knowledge regimes**. Persona and many-shot attacks are typically **black-box**, requiring no gradients or hidden-state access. GCG-family attacks are usually **white-box** or **surrogate-assisted gray-box**. Activation-guided local editing appears to occupy a more privileged regime because it explicitly uses hidden states to guide edits, suggesting access beyond a pure black-box API attacker. UltraBreak, as described, is trained on one or more surrogate VLMs and evaluated for black-box transfer, placing it in a surrogate-based transfer attack regime (1, 2).

A second useful axis is **artifact readability and stealth**. Prompt-level roleplay attacks usually produce readable text but may be easily filtered by semantic safety classifiers. Discrete suffix attacks frequently produce unnatural or nonsensical token strings, which can be highly effective in narrow settings yet operationally brittle and easier to detect. The Wang et al. paper explicitly criticizes token-level attacks as often incoherent or unreadable, and positions activation-guided local editing as retaining readability while improving success (2). UltraBreak similarly treats imperceptible or semantically weighted image perturbations as an attempt to improve universality and transferability under transformations, effectively making the attack both more robust and less conspicuous than brittle pixel-level artifacts (1).

Table I consolidates the 2023–2026 taxonomy into an evaluation-oriented schema.

Family	Primary attack surface	Typical access model	Typical artifact form	Main claimed strengths	Main limitations noted in literature	Representative cited source
Persona / roleplay / reframing	Natural-language prompt	Black-box	Readable text	Low cost, easy deployment, scalable to APIs	Lower stability; model- and policy-specific	2
Multi-turn escalation / Crescendo-like	Conversation state	Black-box	Readable multi-turn dialogue	Can gradually move model into unsafe state;	Higher latency, requires dialogue orchestration	broader 2024–2025 literature; partially contextualized by 2

Family	Primary attack surface	Typical access model	Typical artifact form	Main claimed strengths	Main limitations noted in literature	Representative cited source
				bypasses single-turn filters		
Adversarial suffix optimization (GCG, AttnGCG, AutoDAN-like)	Discrete text tokens	White-box / surrogate	Often unreadable or semi-readable suffixes	High ASR on aligned models in controlled settings	Poorer transferability; brittle to templates and defenses	1
Hidden-state / representation-guided editing	Internal activations + text	White-box / privileged gray-box	Concise edited prompts	Better readability than raw suffix attacks; strong measured ASR	Requires hidden-state access or close surrogate	2
Many-shot / context-budget exploitation	Long context	Black-box	Long demonstration prompt	Exploits in-context learning; often portable	Token-cost heavy; context-window dependent	broader 2024–2026 literature
Universal / transferable multimodal jailbreaks	Image or image-text pair	White-box surrogate to black-box transfer	Adversarial image, sometimes universal	Cross-target and cross-model transfer	Sensitive to architecture, modality pipeline	1

Family	Primary attack surface	Typical access model	Typical artifact form	Main claimed strengths	Main limitations noted in literature	Representative cited source
Tool/agent-mediated exploit chains	Tool invocation policy	Black-box or gray-box	Prompt plus workflow	Targets deployed system behavior, not only text output	Hard to benchmark uniformly; environment-specific	broader 2024–2026 agent security literature
Detection-focused defensive evaluation	Token-level logits / output traces	Defender-side internal telemetry	Risk score / classifier	Stable detection across jailbreak variants	Requires model-side integration	3

An important methodological distinction across these families is whether the attack seeks a **direct refusal bypass** or instead seeks **representation drift** toward benign-seeming internal states. The activation-guided local editing paper is especially important here because it explicitly describes using hidden-state information to “steer the model’s internal representation of the input from a malicious toward a benign one,” indicating a taxonomy shift from surface prompt engineering to attacks on alignment-relevant latent geometry (2). This is significant for evaluation because defenses that rely on surface-level lexical or policy-pattern screening may fail against attacks that preserve benign-looking semantics while altering internal activation trajectories.

A final taxonomic dimension is **optimization objective**. Earlier optimization attacks typically used exact-token cross-entropy objectives. UltraBreak criticizes this approach as brittle and introduces a semantically weighted target objective to reward similar outputs rather than exact strings, thereby smoothing the loss landscape and improving transferability in the multimodal setting (1). This is conceptually relevant beyond VLMs: it suggests that jailbreak evaluation should not categorize methods only by modality but also by whether they optimize exact harmful strings, semantic harmfulness, refusal suppression, or tool-execution outcomes.

Metric design: ASR, refusal bypass, transferability, and reproducibility

The central empirical measure across 2023–2026 jailbreak studies remains **Attack Success Rate (ASR)**, but the literature demonstrates that the term is used inconsistently enough that any rigorous

framework must decompose it into constituent decisions. At minimum, ASR depends on **(a)** what counts as a successful harmful response, **(b)** how refusals are distinguished from partial compliance, **(c)** whether output grading is human, model-based, or rule-based, **(d)** whether repeated attempts are permitted, and **(e)** whether the denominator is harmful prompts, prompt-model pairs, or prompt-attempt pairs. Without this decomposition, cross-paper comparisons are often misleading.

The activation-guided local editing paper reports “state-of-the-art Attack Success Rate” with gains of **up to 37.74% over the strongest baseline**, but the excerpt does not specify in absolute terms whether this refers to percentage points or relative improvement, nor which exact benchmark denominator was used (2). This illustrates a broader reporting problem: improvement claims are common, yet many papers omit enough protocol detail that direct aggregation is difficult. A sound 2023–2026 framework should therefore insist on reporting both **absolute ASR** and **delta over baseline** under identical prompt sets, model versions, templates, temperature settings, and judge criteria.

The VLM literature introduces additional nuance because there are at least four distinct success notions: **prompt-following success**, **unsafe semantic compliance**, **target-string matching**, and **cross-target universality**. UltraBreak explicitly argues that exact-token cross-entropy creates brittle, non-generalizable solutions, and instead uses semantically weighted targets (1). This implies that success should increasingly be measured at the semantic level rather than by exact lexical match. In practice, a robust framework should include:

1. **Refusal Bypass Rate (RBR):** fraction of test cases where the model ceases to refuse and produces materially responsive content.
2. **Unsafe Compliance Rate (UCR):** fraction where the content satisfies a harmfulness or policy-violation criterion.
3. **Task Completion Rate (TCR):** for agent/tool settings, fraction where prohibited actions are successfully executed or meaningfully facilitated.
4. **Universal ASR:** fraction of prompts or targets successfully jailbroken by a single reusable artifact, such as a universal suffix or image.
5. **Transfer ASR:** success on models not used in attack optimization.
6. **Best-of-(N) ASR:** success after (N) retries; this captures practical attacker economics and can greatly inflate observed vulnerability.
7. **Stealth-adjusted ASR:** success conditioned on the attack not being caught by a detector or heuristic filter.

These distinctions are not merely conceptual. They materially affect the apparent ranking of attack families. A readable persona jailbreak might have modest single-shot UCR but high best-of-(N) ASR and near-zero generation cost. A GCG-derived suffix might have excellent white-box RBR but

poor transfer ASR. A universal multimodal jailbreak might have lower exact-target matching but better semantic transfer under image transformations (1, 2).

A second metric issue is **judge stability**. The cited detector paper *SelfGrader: Stable Jailbreak Detection for Large Language Models using Token-Level Logits* is directly relevant because it addresses instability in jailbreak detection and proposes token-level logits as a stable detection signal (3). Its premise implicitly critiques common evaluation pipelines that use another LLM as a judge. Such LLM-as-judge systems can drift across model updates, be sensitive to prompt wording, and themselves be jailbroken. A rigorous evaluation framework should therefore report:

- whether success is judged by humans, rules, or an auxiliary model;
- inter-judge agreement if multiple graders are used;
- the calibration of logits- or classifier-based judges;
- the robustness of judging across paraphrases and output formatting changes.

For reproducibility, the minimum reporting bundle should include: exact model version and date, provider, system prompt if accessible, sampling parameters, harmful prompt set source, prompt template, number of attempts per item, stopping criteria, and grading rubric. Frontier API models change frequently without full version transparency, so **temporal uncertainty** must be explicitly recorded. This is especially important for “frontier-model coverage” claims from 2024–2026, because a paper may report GPT-4o, Claude, Gemini, or open-weight descendants at a particular alignment snapshot that no longer matches later behavior. The UltraBreak excerpt itself notes model-specific anomalies such as weaker transferability to LLaVA-v1.6-Mistral due to weaker underlying alignment and high no-attack ASR (1). This demonstrates why baseline unsafe response rates **without attack** must always accompany ASR.

Table II proposes a standardized metric bundle tailored to 2023–2026 jailbreak studies.

Metric	Definition	Why needed	Common failure mode if omitted
No-attack ASR	Harmful compliance rate on benchmark prompts without adversarial modification	Separates model weakness from attack power	Inflates attack efficacy on weakly aligned models
Single-shot ASR	Success on first attempt per prompt	Captures raw attack potency	Hidden retry budgets make weak attacks appear strong

Metric	Definition	Why needed	Common failure mode if omitted
Best-of-(N) ASR	Success after up to (N) attempts	Models realistic attacker persistence	Not comparable unless (N) and cost are fixed
Transfer ASR	Success on unseen models	Measures portability and ecosystem risk	White-box gains misread as general vulnerability
Universal ASR	One artifact reused across prompts/models	Measures operational scalability	One-off attacks overstated as broadly deployable
Detection-evasion rate	Fraction not flagged by defense	Captures realistic deployment setting	High ASR may collapse when detector is enabled
Readability / perplexity / naturalness	Human or model-based score for artifact plausibility	Distinguishes practical prompts from degenerate strings	Unreadable suffixes may be operationally unusable
Token / query cost	Inputs, optimization steps, or API calls per success	Needed for economics and attacker feasibility	Cheap and expensive attacks falsely treated equally
Judge agreement	Stability across graders / criteria	Increases confidence in success labels	Evaluation may be judge-prompt dependent

For **frontier-model coverage**, a 2023–2026 framework should separate at least four strata:

- **Open-weight instruction-tuned LLMs:** e.g., Llama-family, Mistral-family, Qwen-family, etc.
- **Closed frontier chat LLMs:** e.g., GPT-4-class, Claude-class, Gemini-class models.
- **Vision-language models:** e.g., LLaVA-derived and proprietary multimodal assistants.
- **Agentic systems with tools:** models coupled to browsing, coding, or external actions.

The supplied sources directly support detailed discussion of the VLM stratum and representation-guided text jailbreaks, but only partial support for closed frontier chat models. Therefore, where exact numbers are unavailable in the cited excerpts, the report should distinguish carefully between **documented evidence** and **inference from class behavior**. For example, UltraBreak claims high universality across diverse jailbreak targets and strong transferability from a single surrogate model, directly challenging prior assumptions that multiple surrogates are necessary (1). That is documented. By contrast, any statement that these results “necessarily” imply similar transferability to all proprietary multimodal frontier models would be inference, not established fact.

The role of **cost-normalized evaluation** became increasingly important by 2025. An attack that reaches 80% ASR with 50,000 optimization steps or thousands of API queries may be less operationally threatening than a 35% ASR persona attack requiring only a few handcrafted turns. Likewise, a many-shot attack may achieve high best-of-(N) ASR but be too expensive in long-context token consumption for commodity abuse at scale. Because the activation-guided local editing method claims improved readability and strong efficacy, its position in a cost-normalized ranking may be stronger than raw ASR alone suggests, assuming the privileged hidden-state access is available in the attacker’s environment (2).

A final metric dimension relevant to 2026 is **stability under minor perturbations**. UltraBreak explicitly incorporates randomized transformations and total variation loss to suppress fragile pixel artifacts and make adversarial image patterns transformation-invariant (1). This suggests a broader evaluation principle: every jailbreak paper should test whether success survives benign transformations such as prompt paraphrasing, whitespace changes, system-template variation, image resizing/cropping, and model patch updates. Absent such tests, measured ASR may overstate real-world persistence.

Empirical attack-family matrix: documented efficacy, transfer, cost, and required skill

The cited literature provides partial but important empirical anchors for comparing attack families. Because the request spans persona-based, multi-turn, GCG, many-shot, and tool/agent-mediated attacks, the most defensible approach is to synthesize a matrix that clearly separates **directly cited numbers** from **qualitative evidence** where exact figures are not present in the supplied source excerpts.

The strongest directly cited quantitative claim in the provided material is from *Activation-Guided Local Editing for Jailbreaking Attacks*, which reports state-of-the-art ASR with improvements of **up to 37.74% over the strongest baseline** (2). The description also compares its method to token-level attacks and prompt-level attacks, indicating that it is meant to inherit the scalability of automated

optimization while preserving the readability and coherence advantages of natural-language prompts. The method’s two stages—scenario-based context generation plus hidden-state-guided local editing—are explicitly designed to obscure harmful intent and then redirect internal representation. Empirically, this suggests a family that likely performs well not only on ASR but also on naturalness metrics, though the excerpt does not provide exact naturalness scores.

For optimization-based text attacks, the UltraBreak survey passages summarize the historical role of GCG and successors. The key documented empirical point is negative rather than positive:

transferability remains limited for many discrete-token methods because they operate in token space and on short suffixes (1). This means that even when GCG-style attacks produce high success on the optimization target or surrogate, the ecosystem risk depends on how well they transfer across models and templates. Thus, in a comparative framework, GCG-family methods should be scored strongly on **white-box efficacy** and potentially weakly on **cross-family transfer**, unless a given paper demonstrates otherwise.

The multimodal findings are more concrete on transfer. UltraBreak claims that it **surpasses existing gradient-based methods**, achieves **high universality across diverse jailbreak targets**, and shows **strong transferability even when trained on a single surrogate model** (1). The excerpt further notes a “consistent increase in ASR on black-box models regardless of the chosen surrogate,” with weaker transferability only for LLaVA-v1.6-Mistral due to weaker alignment and high no-attack ASR (1). This is important because it provides one of the clearest 2025–2026 examples that surrogate-trained attacks can generalize without multi-surrogate optimization. It also warns that transfer must be interpreted relative to baseline model weakness.

The literature excerpted also references a provocative 2025 paper subtitle claiming “over 90% success rate against the strong black-box models of GPT-4.5/4o/o1” in the context of a simple attack baseline for VLMs (1). Because this appears only as a citation mention inside another paper and not as full methodological detail, it should be treated carefully. It indicates that by 2025 the field was reporting very high black-box multimodal success rates against strong frontier models, but without the underlying protocol in the provided material, one cannot responsibly generalize the exact magnitude beyond acknowledging the claim’s existence.

Table III therefore distinguishes evidence grade.

Attack family	Directly documented quantitative evidence in provided sources	Transferability evidence	Cost profile	Required skill	Confidence level
Persona / roleplay / reframing	No exact ASR in supplied excerpts	Generally portable across black-box chat models, but unstable	Very low per attempt	Low to moderate	Medium for general characterization, low for exact numbers
Multi-turn escalation / Crescendo-like	No exact ASR in supplied excerpts	Better than single-turn against conversation-managed systems; depends on turn budget	Low to moderate, multiple queries	Moderate	Medium for class behavior, low for exact numbers
GCG / discrete suffix optimization	Historical prominence documented; no exact ASR in supplied excerpts	Explicitly noted as having limited transferability	High compute or query cost for optimization	Moderate to high	High for transferability limitation, low for exact ASR here
Activation-guided local editing	Up to 37.74% gain over strongest baseline	Not fully specified in excerpt; likely better than token-only due to representation guidance	Moderate; privileged internal access may reduce search cost but increases	High in research settings; low deployability without access	High for reported relative gain

Attack family	Directly documented quantitative evidence in provided sources	Transferability evidence	Cost profile	Required skill	Confidence level
			access burden		
Many-shot jailbreaking	No exact ASR in supplied excerpts	Often portable where long contexts exist	High token cost, low compute sophistication	Low to moderate	Medium for class behavior
Universal multimodal jailbreak (UltraBreak)	Claims superiority over prior gradient methods; claims strong universality and transfer	Strong transfer even from single surrogate; exception on LLaVA-v1.6-Mistral	High training cost, low reuse cost once artifact is learned	High to develop, low to deploy	High for qualitative claims, moderate for absent exact figures
“Simple baseline” black-box multimodal attacks	Citation mention of >90% SR vs GPT-4.5/4o/o1	Implied strong transfer to black-box frontier models	Unknown from supplied excerpt	Unknown	Low to medium until protocol is inspected
Tool/agent exploit chains	No exact ASR in supplied excerpts	Depends on tool permissions and environment; transfer often	Can be low or high depending on chain length	Moderate to high	Low from supplied sources

Attack family	Directly documented quantitative evidence in provided sources	Transferability evidence	Cost profile	Required skill	Confidence level
		workflow-specific			

To compare attacker effort more rigorously, it is useful to separate **development cost** from **deployment cost**. GCG, activation-guided editing, and UltraBreak can be expensive to design or optimize, but once a universal suffix, reusable prompt family, or universal image is discovered, marginal deployment cost may be low. This has direct implications for a jailbreak-as-a-service ecosystem: the market will tend to reward attacks with high up-front R&D cost but strong reusability. UltraBreak’s emphasis on universality and transferability is therefore economically significant, because it transforms a research artifact into something more compatible with repeated operational use (1).

The same economic logic explains why many real-world attackers historically favored **persona and roleplay** prompts despite lower nominal ASR in controlled benchmarks: they are cheap to iterate, require no model internals, and can be rapidly adapted to new safety policies. However, as providers deploy stronger refusal systems and policy-template hardening, multi-turn and many-shot variants become more attractive because they exploit conversational consistency and in-context learning rather than only surface instruction parsing. While the supplied citations do not provide explicit Crescendo numbers, this family should be evaluated on **turn efficiency**: ASR as a function of dialogue turns. A method that needs ten turns to succeed is qualitatively different from one that succeeds in two, even if final ASR is similar.

For frontier-model coverage, the current evidence from supplied sources is strongest for:

- **Open or research-access VLMs** via UltraBreak and prior VAJM/ImgJP/UMK comparisons (1).
- **Representation-guided LLM jailbreak settings** via activation-guided local editing (2).
- **Defender-side jailbreak detection** via token-level logits in SelfGrader (3).

Coverage is weaker in the supplied material for exact model-by-model ASR tables on proprietary frontier chat-only LLMs. Accordingly, any comprehensive empirical matrix for closed models must

note incomplete documentation in the present source set rather than fabricate precision. The right methodological response is to normalize all claims into the evidence categories shown in Table IV.

Evidence category	Meaning	Example from supplied sources
A: Exact reported metric and comparator	Quantified claim with explicit delta or rate	“up to 37.74% over the strongest baseline” for activation-guided local editing (2)
B: Strong qualitative empirical claim	Clear experimental result without exact number in excerpt	UltraBreak “surpasses existing gradient-based methods” and has “strong transferability” from one surrogate (1)
C: Contextual survey statement	Literature summary about family behavior	GCG improved but “transferability remains limited” (1)
D: Citation-only mention	Numeric or strong claim appears only in passing	“over 90% success rate against ... GPT-4.5/4o/o1” in a reference mention (1)
E: Inference	Not directly evidenced in supplied excerpts	Economics of commoditized suffix resale markets

This evidence grading is essential because the request asks for attack success rates “across specific frontier models.” In the present source bundle, some frontier-model references are specific but not fully elaborated; therefore, responsible reporting must keep exactness bounded by the available documentation.

Frontier-model coverage and what defenses each family bypasses

A 2023–2026 evaluation framework should treat “frontier-model coverage” as more than a list of models tested. It must capture which **defense stack** is present on each model, because attack success is jointly determined by base model alignment, system prompts, input classifiers, output filters, tool policies, and monitoring layers. The same family can appear highly effective against one deployment and much weaker against another because it bypasses only certain layers.

The activation-guided local editing paper is explicit that prompt-level attacks and token-level attacks each have characteristic drawbacks, and its proposed two-stage method effectively bypasses

them by combining contextual obfuscation with representation steering (2). The defenses it appears designed to evade include:

1. **Surface harmful-intent detectors**, by rephrasing malicious queries and embedding them in scenario context.
2. **Template-triggered refusal heuristics**, by making the final prompt more semantically indirect and benign-seeming.
3. **Robustness limitations of prompt-only alignment**, by steering hidden-state representations toward regions associated with benign inputs.

Thus, for frontier text models, a key distinction is between systems protected mainly by lexical/policy filters and those with stronger latent or output-stage safety mechanisms. If a jailbreak works by changing internal representation while preserving benign surface form, models relying heavily on prompt-side classifiers may be more vulnerable than those using deeper output-monitoring or logit-level detectors.

The SelfGrader paper is relevant precisely because it proposes **token-level logits** as a stable signal for jailbreak detection (3). This suggests a defense class aimed at catching not just explicit unsafe words but anomalous generation dynamics associated with jailbreak success. Such defenses could, in principle, improve robustness against:

- unreadable adversarial suffixes;
- semantically indirect jailbreak completions;
- surface-benign prompts that nonetheless induce risky token distributions.

However, because the excerpt provides only the paper's title and framing, one should not overstate demonstrated coverage. Still, its existence indicates a 2026 trend away from brittle string filters toward model-internal telemetry defenses.

The VLM literature provides the clearest mapping between attack families and bypassed defenses. UltraBreak is designed to overcome the limitations of prior multimodal jailbreaks that transfer poorly under architecture changes or minor transformations. To do so, it uses randomized transformations during training to force transformation-invariant patterns, total variation loss to avoid fragile artifacts, and semantically weighted targets rather than exact cross-entropy strings (1). The defenses it thereby pressures include:

1. **Image preprocessing defenses** such as resizing or mild transformations, because the optimization explicitly trains for invariance.
2. **Architecture diversity**, because the method targets broadly recognized jailbreak-inducing features rather than model-specific quirks.

3. **Target-string brittleness**, by optimizing semantic compliance instead of exact outputs.

This is strategically important for frontier multimodal models: if a universal image can transfer across black-box systems from a single surrogate, the defender cannot rely solely on proprietary architecture secrecy or light preprocessing. The UltraBreak excerpt specifically states that a consistent increase in black-box ASR occurs regardless of chosen surrogate, though LLaVA-v1.6-Mistral is an exception associated with weaker alignment ([1](#)).

Table V maps major attack families to commonly bypassed defenses.

Attack family	Defensive layer commonly bypassed	Mechanism of bypass	Limits of bypass
Persona / roleplay	Static refusal templates, keyword rules	Reframes task as fictional, evaluative, or role-bound	Strong semantic moderators may still catch unsafe intent
Multi-turn escalation	Single-turn moderation and per-message heuristics	Gradually shifts conversation state and commitment	Conversation-level monitors can detect escalation patterns
Many-shot	Direct instruction filters	Uses in-context examples to normalize unsafe behavior	Long-context truncation or demo-aware policy checks can reduce effect
GCG / adversarial suffix	Standard alignment tuning and prompt filters	Exploits token-level vulnerabilities in decoder behavior	Often brittle to paraphrase, template changes, and detectors
Activation-guided local editing	Surface semantic filters; prompt-side moderation	Moves latent representation toward benign region while preserving attack objective	Harder to execute without internal access
Universal multimodal jailbreak	Modality separation, light image transforms, architecture secrecy	Learns transformation-robust, semantically effective visual triggers	May fail under strong multimodal sanitization or robust retraining
			Tool-use policies and action verification can

Attack family	Defensive layer commonly bypassed	Mechanism of bypass	Limits of bypass
Tool/agent exploit chains	Text-only safety filters	Splits harmful goal across allowed substeps or tools	block end-to-end completion
Best-of-(N) search	Deterministic policy responses	Exploits stochasticity and response variance	Rate limiting and adaptive throttling reduce attack budget

In terms of **frontier-model coverage**, the supplied sources support the following observations.

First, for **vision-language frontier models**, the literature has clearly moved from model-specific visual jailbreaks toward claims of universality and transferability. The explicit mention of strong black-box transfer from a single surrogate and of prior works failing under minor architectural changes shows that transfer has become a central evaluation criterion by 2025–2026 (1).

Second, for **aligned LLM chat models**, the main frontier trend reflected in the provided papers is a shift from hand-authored prompt attacks to methods that either optimize suffixes or exploit hidden-state information to recover readability and stronger ASR. The Wang et al. paper’s criticism of token-level incoherence and prompt-level manual burden neatly captures the design pressure on new text attacks: increase automation without sacrificing semantic plausibility (2).

Third, for **defender-side mechanisms**, token-level logit detection suggests that frontier providers are moving toward internal anomaly scoring rather than relying solely on visible text moderation (3). Such defenses are especially relevant against attacks that preserve surface plausibility.

A subtle but important point is that **weakly aligned models can confound transfer measurements**. UltraBreak’s exception on LLaVA-v1.6-Mistral is attributed to the model’s high no-attack ASR (1). Thus, when a paper claims transfer across frontier models, the result should be stratified by baseline alignment strength. Otherwise, a family may appear broadly transferable simply because some targets are intrinsically permissive.

From a benchmarking perspective, each frontier model should therefore be annotated with:

- base no-attack ASR,
- safety stack type,
- context-window size,
- multimodal preprocessing behavior,
- tool permissions if any,

- version date.

Only then can cross-family comparisons be interpreted. An unreadable suffix that succeeds on a weakly aligned open-weight model but fails on closed frontier APIs may tell us less about ecosystem risk than a readable multi-turn attack with moderate but consistent success across multiple closed models.

Defensive mechanisms, benchmark design, and the economics of adversarial prompt development

The literature cited in the prompt reveals a defense landscape that can be grouped into **pre-input moderation**, **model-side alignment**, **output moderation**, **internal-telemetry detection**, and **deployment controls** such as rate limits and tool gating. The strategic significance of jailbreak research between 2023 and 2026 is that attack families have progressively adapted to bypass each layer rather than only the base alignment model.

Pre-input moderation includes keyword filters, semantic classifiers, and pattern matching for known jailbreak templates. Persona, many-shot, and activation-guided attacks pressure this layer by making harmful intent less explicit. The Wang et al. method is almost a direct response to this defense class: its first stage “rephrases the original malicious query to obscure its harmful intent,” and the second stage uses hidden states to shift the internal representation toward benignness (2). This means that lexical or even shallow semantic filters may increasingly miss attacks that are contextually obfuscated yet representation-effective.

Model-side alignment includes supervised fine-tuning, RLHF, constitutional tuning, refusal templates, and safety-specific decoder behavior. GCG-class attacks directly target this layer by searching for token strings that exploit the aligned model’s residual vulnerabilities. The VLM literature’s summary that GCG introduced the first optimization-based jailbreak and that follow-up works enhanced it but still struggled with transferability suggests that aligned decoders remain vulnerable to optimization pressure, though these vulnerabilities may be idiosyncratic rather than universal (1).

Output moderation can block explicit harmful completions even if the base model begins to comply. However, attacks that optimize for semantically weighted or oblique outputs may still convey prohibited content while avoiding exact-match filters. UltraBreak’s critique of exact-token objectives is relevant here: by rewarding semantically similar outputs, it effectively optimizes for the behavior defenders care about, not merely for a target string ([1](https://

Persona-Based, Manual, and Many-Shot Jailbreaking: Empirical Effectiveness, Operational Burden, Transfer, and Safeguard Evasion

Scope note and differentiation from prior sections

The earlier report already established the overall taxonomy, metric caveats, and a high-level family matrix. This section therefore does **not** restate generic ASR methodology, broad defense taxonomy, or unrelated multimodal/tool-chain material. Instead, it narrows to three closely related but operationally distinct prompt-centric families that are disproportionately important in practice: **persona/manual jailbreaks**, **human-crafted compositional prompts**, and **many-shot or in-context demonstration jailbreaks**. The emphasis here is on:

- 1) **measured effectiveness from in-the-wild and benchmarked sources**,
- 2) **execution cost and attacker skill**,
- 3) **cross-model portability**, and
- 4) **which prompt-level safeguards these attacks are documented to defeat or pressure**.

Where evidence is direct, it is labeled as such; where ecosystem implications are inferred from repository and review evidence, that is stated explicitly ([1](#), [2](#)).

Operational distinctions among the three families

Although these families are often lumped together as “prompt jailbreaks,” the literature supports useful distinctions.

- **Persona-based/manual jailbreaks** rely on human-authored role framing, authority inversion, policy reinterpretation, or emotional/social manipulation. Typical examples in the wild include “Do Anything Now” style roleplay, simulation framings, dual-response formats, and compliance-inducing personas. Their core advantage is low technical barrier and rapid adaptation by human attackers. Their core weakness is variance: many are brittle, platform-specific, and sensitive to moderation updates. Evidence for their prevalence comes primarily from in-the-wild prompt collection efforts rather than only lab benchmarks ([1](#)).
- **Manual but structured compositional jailbreaks** are more deliberate benchmarked attacks that exploit instruction decomposition or semantic obfuscation while still being human-readable. The clearest supplied example is the **Compositional Instruction Attack (CIA)**, which achieves very high success rates on major aligned text models and is specifically evaluated against an intent-focused defensive layer ([2](#)).

- **Many-shot / in-context demonstration jailbreaks** use repeated demonstrations, often harmful exemplars, to induce the model to imitate unsafe behavior despite safety fine-tuning. This family is distinct because its attack resource is chiefly **context budget** rather than clever wording alone. The review source explicitly identifies **In-Context Demonstrations Attack (ICA)** and notes that harmful demonstrations can materially increase jailbreak success, while a defense based on refusal exemplars can counter it (2).

These distinctions matter empirically. A low-skill persona jailbreak can be cheap and abundant in real-world ecosystems even if single-prompt reliability is modest. A carefully engineered compositional attack may be more reproducible and benchmark-dominant. A many-shot jailbreak may be expensive in tokens yet require comparatively little search sophistication. This “cheap-but-variable” versus “costlier-but-systematic” split is central to both attacker operations and defender prioritization.

In-the-wild prevalence: manual jailbreaks as the dominant observable ecosystem artifact

The strongest supplied evidence on what attackers actually circulate publicly is the [CCS 2024 JailbreakHub repository](#), which reports a corpus of **15,140 prompts** collected from **Reddit, Discord, websites, and open-source datasets** over **December 2022 to December 2023**, including **1,405 jailbreak prompts**; an earlier version documented **6,387 prompts** with **666 jailbreak prompts** through May 2023 (1, 3). Even without reproducing their entire categorization taxonomy here, the dataset’s provenance strongly indicates that **manual prompt sharing, remixing, and iterative community adaptation** were major vectors of jailbreak dissemination during the first large-scale public deployment cycle of aligned chat models.

This repository evidence does not by itself prove that persona-style prompts had the highest ASR among all families. However, it does establish three operationally important facts.

First, **manual jailbreak development was abundant enough to sustain a visible public prompt-sharing ecosystem**. The scale—1,405 jailbreak prompts in the larger release—implies that adversarial prompt development was not restricted to researchers or highly technical attackers. Second, **prompt sharing occurred across heterogeneous social and repository platforms**, which is consistent with low entry cost and rapid mutation of templates. Third, the repository’s own framing—“the largest collection of in-the-wild jailbreak prompts”—shows that empirical security work increasingly had to account for attacker behavior beyond curated benchmarks (1).

From a defense standpoint, this is critical because prompt-level safeguards are often evaluated against synthetic benchmarks or optimizer-generated attacks, while manual persona prompts reflect

deployed adversarial adaptation under realistic user constraints. The real-world ecosystem therefore acts as a selection mechanism: prompts that are easy to explain, easy to copy, and easy to mutate can persist even if they are less elegant than research attacks.

Benchmarked effectiveness available in the supplied literature

The supplied review does not enumerate a complete ASR table specifically for persona prompts collected in the wild, but it does provide benchmark evidence for two neighboring classes highly relevant to manual and many-shot attacks:

- **Compositional Instruction Attack (CIA):** >95% ASR against GPT-4, GPT-3.5, and Llama2-70B. It is described as a jailbreak method evaluated against an **Intent-Based Defense (IBD)**, and the review states that the defense **reduces ASR by >74%** by identifying the malicious intent beneath the composed instructions (2).
- **In-Context Demonstrations Attack (ICA):** the review states that harmful demonstrations **increase jailbreak success on various aligned LLMs**, while **In-Context Defense (ICD)** using refusal examples can counter the attack by reducing success rates. The supplied excerpt does **not** give a single unified ASR percentage, so any precise numeric claim beyond this should be avoided here (2).

These are the clearest direct benchmark anchors for this section. They should not be conflated: CIA is evidence that **manual, readable, semantically structured jailbreaks can attain very high success on frontier and near-frontier aligned chat models**. ICA is evidence that **demonstration-driven context manipulation is a viable and distinct pathway to jailbreak**, even if the exact magnitude is omitted in the supplied snippet.

Empirical comparison table

Table I consolidates only the evidence directly supported by the supplied sources and flags where the literature is stronger on qualitative than quantitative detail.

Attack family	Specific documented attack	Model coverage in supplied sources	Reported effectiveness	Prompt-level safeguard or defense pressured	Evidence quality in supplied sources
	Publicly shared jailbreak	ChatGPT-focused	No single ASR figure in	Generic instruction-	High for prevalence,

Attack family	Specific documented attack	Model coverage in supplied sources	Reported effectiveness	Prompt-level safeguard or defense pressured	Evidence quality in supplied sources
Persona/ manual in the wild	prompts collected in JailbreakHub	ecosystem dataset; public prompt-sharing platforms	supplied excerpt; strong prevalence evidence with 1,405 jailbreak prompts out of 15,140 prompts collected	following safety layers in deployed chat systems; exact bypass mechanism varies by prompt	moderate for efficacy because ASR values are not in the provided snippet (1)
Human-crafted compositional prompt jailbreak	Compositional Instruction Attack (CIA)	GPT-4, GPT-3.5, Llama2-70B	>95% ASR	Intent-Based Defense (IBD) aimed at harmful-intent detection; defense reportedly reduces ASR by >74%	High (2)
Many-shot / demonstration-based jailbreak	In-Context Demonstrations Attack (ICA)	Various aligned LLMs	Harmful demonstrations increase success; no exact unified ASR in supplied excerpt	In-Context Defense (ICD) using refusal examples	Moderate-to-high for existence and defense interaction, limited for exact

Attack family	Specific documented attack	Model coverage in supplied sources	Reported effectiveness	Prompt-level safeguard or defense pressured	Evidence quality in supplied sources
					percentages (2)
Automated but still human-readable prompt mutation baseline for comparison	GPTFuzz	ChatGPT, LLaMA-2, Vicuna, others	>90% success rates across multiple LLMs	Judgment-model-guided refinement pressures conventional prompt moderation and alignment	High, but included here only as comparator for manual-scale prompting rather than focal family (2)
Automated benchmark comparator for query efficiency	GAP	GPT-3.5	92% ASR , using 54% of the prior queries ; PromptGuard accuracy improved from 5.1% to 93.89% on Toxic-Chat after tuning	Baseline prompt detection and model-side moderation	High, included as cost comparator rather than focal family (2)

The comparison is useful because it shows that **human-readable and semantically coherent jailbreaks are not merely weak “social engineering” curiosities**. In at least one benchmarked form, they reach **>95% ASR** on major aligned models, placing them in the same general empirical range as many highly optimized attack families.

Measured Effectiveness of Manual and Persona Jailbreaks in the Wild Versus Benchmarked Settings

Why in-the-wild evidence and benchmark evidence diverge

A central empirical challenge in this area is that **publicly circulated persona jailbreaks are overrepresented in ecosystem visibility**, while **benchmarked compositional attacks are overrepresented in reproducible ASR reporting**. This asymmetry is important for security analysis.

The JailbreakHub repository demonstrates wide public circulation of jailbreak prompts, but the excerpt supplied here is primarily about **dataset scale and provenance**, not model-by-model performance tables (1). By contrast, the 2025 review source summarizes benchmark results such as CIA with explicit model coverage and ASR (2). Therefore, the most rigorous reading is:

1. **In-the-wild evidence strongly supports prevalence and adversarial labor supply**, especially for persona/manual prompts.
2. **Benchmarked evidence more clearly supports measured effectiveness**, especially for structured attacks such as CIA and ICA.
3. It is reasonable—but still partly inferential—to view persona/manual prompting as the operational substrate from which more formalized benchmark attacks draw design motifs.

This distinction matters because defenders may underinvest in low-tech prompt attacks if they prioritize only optimizer-generated or white-box literature. The ecosystem data suggest that such a choice would be strategically mistaken.

Manual/persona prompts: what can be said empirically from the supplied record

The supplied sources support a cautious but still substantive empirical characterization of persona/manual jailbreaking:

- The attack ecosystem is **large and persistent**. The collection of **1,405 jailbreak prompts** from a broader corpus of **15,140 prompts** across four data sources is strong evidence that manual jailbreaks were not anecdotal edge cases but a sustained, distributed practice during the 2022–2023 period (1).
- The ecosystem was **iterative and socially transmissible**. Because prompts were drawn from Reddit, Discord, websites, and open datasets, it is highly likely that many jailbreaks were

copied, adapted, and recombined by users with modest technical sophistication. This claim is an inference from platform mix and collection design, but it is a strong one.

- Public examples referenced in the supplied literature include well-known roleplay styles such as DAN-era prompts, reflected indirectly through references in later papers discussing “in-the-wild jailbreak prompts” and public platform posts (4). The excerpt from the black-box paper’s references is incomplete, so it should not be treated as a primary effectiveness source; however, it does corroborate that the academic literature sees public manual jailbreaks as a major antecedent.

What the supplied sources **do not** allow is a definitive percentage claim such as “persona-based prompts achieve X% ASR across frontier models.” Because that precise number is not present in the prompt materials, any such figure would overstate the evidence. The correct empirical posture is that **manual persona prompts are demonstrably widespread and operationally important, but the strongest provided ASR figures pertain to benchmarked compositional/manual variants rather than generic in-the-wild personas.**

Structured manual prompts: CIA as the strongest benchmarked evidence in this subset

The **Compositional Instruction Attack (CIA)** is especially important because it bridges the gap between crude persona prompts and optimizer-generated attacks. It remains **human-readable**, targets **major aligned language models**, and achieves **>95% ASR** against **GPT-4, GPT-3.5, and Llama2-70B** in the supplied review (2).

This result has several implications.

First, it demonstrates that **manual or semi-manual semantic decomposition can be sufficient to overwhelm mainstream safety alignment**, even on models that had already undergone substantial post-training safety tuning. Second, because the target set spans both OpenAI models and an open-weight family, CIA provides direct evidence that this class is **not restricted to one vendor’s alignment idiosyncrasies**. Third, the attack’s success against an evaluated intent-focused defense implies that the prompt does not merely hide harmful strings at the surface level; rather, it exploits the model’s ability to recombine apparently benign or distributed instructions into a harmful latent objective.

The reported **>74% ASR reduction under Intent-Based Defense** is equally informative (2). It indicates that semantic intent detection can materially blunt this attack family, yet it does **not** indicate perfect elimination. Put differently, the defense confirms the nature of the threat but not its

full containment. For a security practitioner, this means that a benchmark-winning manual attack can remain consequential even after a substantial defensive gain.

Many-shot jailbreaks: demonstration effects are real even when exact percentages are sparse

The review's treatment of **In-Context Demonstrations Attack (ICA)** is concise but important: harmful demonstrations can increase jailbreak success on **various aligned LLMs**, while **In-Context Defense (ICD)**—adding refusal examples—can reduce that success (2). This establishes three things with confidence.

1. **The mechanism is imitation pressure through context**, not merely direct instruction override.
2. **Alignment can be context-fragile**: preceding examples shift behavior enough to elevate harmful compliance.
3. **Counter-conditioning in the prompt itself can help**, meaning the failure mode is at least partly prompt-local rather than entirely embedded in model weights.

The absence of a quoted unified ASR percentage in the supplied snippet does not diminish the family's importance. In fact, the absence highlights a practical challenge: many-shot jailbreaks can be highly sensitive to **context window**, **demonstration ordering**, **model sampling setup**, and **whether the evaluation permits long prompts**. Consequently, the most robust statement supported here is that **many-shot attacks are empirically effective enough to merit a dedicated defensive strategy**, not that they have one canonical percentage.

Relative effectiveness versus automated comparators

Although this section is focused on persona/manual/many-shot attacks, the supplied review includes automated comparators that sharpen interpretation.

- **GPTFuzz** reports **>90% success rates across multiple LLMs**, scaling better than manual template crafting (2).
- **GAP** achieves **92% ASR** on **GPT-3.5** while using only **54% of the required queries** compared with prior querying needs (2).

These results imply that manual and many-shot attacks should not be dismissed because automated attacks also perform well. Rather, they suggest a continuum:

- **manual persona prompts**: easiest to create and share, but noisier;

- **structured manual/compositional prompts:** still readable, but much more benchmark-effective;
- **many-shot prompts:** often token-expensive but conceptually simple;
- **automated prompt search/fuzzing:** operationally more systematic and often query-efficient.

CIA’s **>95% ASR** is particularly striking in that it competes numerically with automated families while remaining semantically intelligible. This makes it operationally valuable: readable attacks are easier to distribute and adapt than opaque optimizer outputs.

A more nuanced effectiveness matrix

The practical question is not simply “which family has the highest ASR,” but “under what attacker constraints does each family remain effective?” Table II reframes the evidence in those terms.

Constraint regime	Persona/ manual prompts	Compositional manual prompts (e.g., CIA)	Many-shot demonstrations
Very low attacker skill	Strongly favorable; easiest entry point	Less favorable; requires attack design sophistication	Moderate; attacker must understand example construction
Low token budget	Favorable	Favorable to moderate	Often unfavorable, since success depends on long context
Need for human readability	Strongly favorable	Strongly favorable	Moderate; long demonstrations can look conspicuous
Need for benchmark-grade repeatability	Weak to moderate	Strong	Moderate, but sensitive to context setup
Need to pressure intent-level defenses	Moderate	Strong, explicitly shown against IBD	Moderate; depends on whether demonstrations preserve harmful latent intent
	Strong	Strong	

Constraint regime	Persona/ manual prompts	Compositional manual prompts (e.g., CIA)	Many-shot demonstrations
Need for easy community sharing			Moderate; length reduces convenience

This framing is not a substitute for model-level ASR tables. Rather, it explains why persona/manual and many-shot attacks remain strategically important even when automated methods often achieve stronger formal optimization results.

Execution Cost, Query Burden, and Attacker Skill Requirements

This section’s distinction from earlier cost discussions

Previous content in the larger report discussed attack-family cost at a broad level, including optimizer-based and multimodal attacks. This section differs by focusing specifically on **human labor, prompt engineering effort, token expenditure, and practical deployment burden** for persona/manual and many-shot attacks. It emphasizes the attacker economics visible from the supplied in-the-wild repository and the benchmark review, rather than repeating generic “high vs. low cost” labels.

Persona/manual attacks: minimal compute, non-trivial craft knowledge

The most salient property of manual persona jailbreaks is that they are **compute-light but not necessarily insight-free**. A user can attempt them without model gradients, specialized optimization code, or large query budgets. The cost profile is therefore dominated by:

- **human ideation time,**
- **trial-and-error refinement,**
- **knowledge of current refusal patterns, and**
- **copying or adapting from community-shared templates.**

The JailbreakHub dataset implies a large supply of reusable prompt artifacts ([1](#)). This means many attackers can avoid the most expensive component—original attack development—by borrowing extant prompts. The resulting cost structure is unusual compared with classical cyber exploitation:

the marginal deployment cost can approach zero once the social ecosystem has produced enough reusable prompt stock.

From a defender's perspective, this makes persona/manual attacks dangerous despite their uneven reliability. Even if any single prompt degrades after moderation updates, the attacker's replacement cost is often trivial. New variants can be generated by paraphrase, persona substitution, or prompt chaining. This is one reason public jailbreak communities matter: they effectively amortize research and development across many users.

Skill model for persona prompts

A useful distinction is between **foundational skill** and **operational skill**.

- **Foundational skill:** understanding that the model can be induced to roleplay, split responses, reinterpret policies, or prioritize user-framed fictional contexts.
- **Operational skill:** recognizing when a model update has neutralized a prompt and quickly mutating the framing to restore compliance.

The in-the-wild dataset's scale suggests that the foundational skill threshold is low enough for many users, while operational skill is distributed socially through shared prompts and discussion channels (1). Thus, persona attacks have a **low median skill requirement**, even if a small set of prolific authors likely contribute disproportionately strong prompt innovations.

Compositional manual attacks: higher design sophistication, still modest infrastructure cost

The **Compositional Instruction Attack** represents a more advanced human-authored class. Its **>95% ASR** indicates that strong effectiveness does not require heavy optimization infrastructure when the attack is well designed (2). However, compared with ad hoc persona prompts, CIA likely imposes higher attacker costs in at least four ways:

1. **Prompt architecture design:** the attacker must understand how to distribute or transform the harmful objective into compositional instructions that remain semantically recoverable by the model.
2. **Model behavior experimentation:** attack construction likely requires iterative testing against specific refusal styles.
3. **Evaluation discipline:** to reach reproducible benchmark performance, attackers need a systematic prompt set and grading method.
4. **Semantic subtlety:** the attack must preserve harmful intent while avoiding simple triggers or naive moderation heuristics.

Even so, infrastructure cost remains modest relative to optimizer-driven attacks. There is no indication in the supplied excerpt that CIA depends on gradients, surrogate training, or large-scale search. This matters operationally: **the barrier to a highly effective attack can remain within reach of advanced prompt engineers rather than ML specialists.**

Many-shot attacks: token-expensive, cognitively simple, often organizationally convenient

Many-shot jailbreaks invert the cost profile. They can be **conceptually simple**—show the model repeated harmful exemplars and prompt it to continue in the induced pattern—but they may be **token-expensive** and therefore financially costly on commercial APIs or context-constrained interfaces.

Their cost components include:

- constructing or collecting suitable demonstrations,
- fitting them within the target model’s context budget,
- potentially adjusting order and content for stability, and
- paying for larger prompt token counts on each attempt.

This creates a tradeoff absent in short persona prompts. A manual roleplay jailbreak might be cheap to try repeatedly. A many-shot jailbreak may require fewer clever semantics but consumes more context and can become costly at scale, particularly against premium frontier APIs with long-context pricing.

The supplied review’s statement that harmful demonstrations increase jailbreak success, while refusal demonstrations can reduce it, also implies that attackers and defenders are both competing over **scarce context real estate** (2). In deployments where system prompts, retrieval context, or tool traces already occupy significant tokens, many-shot attack headroom may be constrained. Thus, many-shot attacks are often attractive where long prompts are tolerated, but less so where interfaces or middleware aggressively compress user input.

Query burden and the contrast with automated search

The review’s **GAP** result—**92% ASR at 54% of the prior query cost**—is not directly a manual attack result, but it illuminates the cost landscape (2). It shows that one reason automated jailbreak search attracts research attention is not merely higher ASR, but **better query efficiency at scale**. Manual and many-shot attacks differ:

- **persona prompts** often need little infrastructure but may require multiple human retries;

- **many-shot prompts** may need fewer semantic rewrites but incur larger token charges;
- **automated search** can reduce queries or systematically optimize prompt variants once the pipeline exists.

Thus, the choice of attack family depends on attacker economics. Hobbyist or opportunistic attackers may prefer manual prompts because they are practically free. More organized actors or benchmarking researchers may prefer automated pipelines that trade development effort for repeatability and lower per-success query cost.

Economic decomposition: development versus deployment

For these prompt-centric families, separating **development cost** from **deployment cost** is especially important.

1) Persona/manual prompts

- **Development cost:** low to moderate; mostly human ideation.
- **Deployment cost:** very low.
- **Reuse potential:** high, especially through social sharing.
- **Skill concentration:** innovation concentrated in a small group; deployment diffused to many users.

2) Compositional manual prompts

- **Development cost:** moderate; requires stronger prompt-engineering insight.
- **Deployment cost:** low.
- **Reuse potential:** high if prompts remain readable and adaptable.
- **Skill concentration:** higher than persona prompts, but still far below white-box optimization.

3) Many-shot demonstrations

- **Development cost:** low to moderate if example sets are available.
- **Deployment cost:** moderate to high because of token length.
- **Reuse potential:** moderate; examples can be copied but may need adaptation to model and context constraints.
- **Skill concentration:** lower technical sophistication, but stronger need for operational awareness of context limits.

This decomposition helps explain why the real-world ecosystem can support a “jailbreak-as-a-service” style dynamic even without a literal formal service marketplace visible in the supplied

sources. A small number of capable prompt authors can generate reusable prompts; a much larger downstream user base can deploy them at near-zero marginal cost. The JailbreakHub collection is a concrete artifact of this economic structure (1).

Defender implications of low-skill attack availability

The prevalence of shared manual prompts means that defenders should not model adversaries only as high-skill ML specialists. The more immediate adversarial reality is often:

- low-skill end users,
- assisted by community-distributed prompt templates,
- operating on short feedback loops,
- with minimal financial cost per attempt.

This is why benchmarked high-ASR prompt attacks matter operationally even when they are not fully automated. If an attack like CIA is semantically transparent and reproducible, it can potentially diffuse into the broader prompt-sharing ecosystem much more easily than a brittle, unreadable suffix or a research-only white-box artifact. The defender's challenge is therefore not only to stop individual prompts, but to reduce the return on **mass prompt variation**.

Cross-Model Portability and Family-Specific Transfer Dynamics

Why transfer must be treated differently for these attack families

Earlier parts of the larger report discussed transferability as a general metric. This section narrows the issue to the transfer properties of **manual persona**, **compositional manual**, and **many-shot** attacks. These families behave differently from optimized suffixes or multimodal triggers because they rely on **shared instruction-following priors**, **common RLHF-style alignment patterns**, and **generic imitation behavior** rather than model-specific token quirks.

As a result, their transfer often depends less on exact architecture similarity and more on whether different models were trained to respond to broadly similar conversational cues.

Direct evidence: CIA transfers across vendor and model family boundaries

The strongest direct transfer evidence in the supplied sources is that **CIA** achieves **>95% ASR** on **GPT-4**, **GPT-3.5**, and **Llama2-70B** (2). This is notable because these models differ in provider,

training data, size, and alignment implementation. Even without a full transfer matrix, the result strongly suggests that the attack is exploiting **generalizable alignment failure modes** rather than one narrow platform bug.

The significance is twofold.

First, the family appears to exploit **shared safety abstractions**: models are trying to be helpful, compositional, and cooperative, and the attack distributes harmful intent across a structure that those abstractions reconstruct. Second, this means defensive lessons from one platform may generalize. If intent-level reconstruction helps on one model family, it is at least plausible that similar semantic defenses could help on others.

Manual persona prompts: likely broad but unstable transfer

For in-the-wild persona/manual prompts, the supplied sources offer stronger evidence of widespread use than formal transfer matrices. Still, the ecosystem data support a reasonable transfer assessment.

Because the JailbreakHub prompts were collected across public communities centered on widely used chat models, and because the same social spaces historically discuss adapting prompts across systems, it is plausible that many persona jailbreaks were **designed for portability or rapidly ported by users** ([1](#)). Their transfer basis is typically:

- roleplay or simulation framing,
- authority or task reframing,
- dual-output formatting,
- lexical softening or indirection,
- social engineering of helpfulness norms.

These cues are not vendor-specific. They target generic conversational alignment heuristics found across many chat-tuned models. However, persona prompts also tend to be **unstable under moderation updates** because they are often easy to recognize once a provider has seen them at scale. Therefore, their transfer profile is best described as **broad but volatile**.

This differs from CIA. A structured compositional attack can be broad and also benchmark-stable. A generic persona prompt may be broad but much more ephemeral. That makes public prompt ecosystems powerful yet noisy: they excel at spreading attacks faster than vendors can label them, but individual prompts may decay quickly.

Many-shot transfer: portability through generic imitation behavior

The review's ICA summary refers to **various aligned LLMs**, which is already a portability signal (2). The mechanism of transfer is straightforward: many aligned chat models are highly sensitive to in-context examples and exhibit strong continuation behavior. Therefore, a many-shot attack need not discover model-specific hidden triggers; it can instead exploit a near-universal property of instruction-tuned LLMs—**behavioral adaptation to demonstrations**.

This gives many-shot jailbreaks a special form of portability:

- If a model has sufficient context length,
- and is trained to follow demonstrations,
- and does not strongly compartmentalize harmful examples from desired outputs,

then harmful few-shot or many-shot prompts may transfer reasonably well.

Their main portability constraints are usually not architecture but **prompt budget, conversation format, and system-prompt precedence**. A compact roleplay prompt may fit almost anywhere; a long many-shot prompt may fail simply because the deployment environment truncates or reorders context.

Transfer versus detectability tradeoff

An important but underappreciated feature of these families is that **transferable attacks may also be easier to detect if they become standardized**.

- A famous persona prompt can spread across many models, but once it becomes recognizable, providers can patch against its surface form.
- A compositional attack like CIA may remain transferable if it is semantically diverse enough to resist signature-based blocking.
- A many-shot attack may transfer well in principle, but long harmful demonstrations are easier for moderation systems to inspect than short obfuscated prompts.

Thus, transfer should not be equated with persistence. Manual prompts often transfer because they appeal to common conversational priors. They may still degrade quickly once defenders harden around popular templates.

Comparison with automated readable attacks

The supplied review's coverage of **AutoDAN** and **GPTFuzz** is useful here because these methods preserve semantic coherence while improving transfer. **AutoDAN** is described as producing

stealthy, semantically meaningful prompts that outperform prior methods in **cross-model transfer and stealth**, and **GPTFuzz** reports **>90% success rates across multiple LLMs** (2). This matters for interpreting manual families.

Historically, one might assume a tension between readability and transfer: human-readable prompts are easier to patch, while optimized gibberish may discover stranger, more universal weaknesses. The supplied evidence complicates that assumption. CIA, AutoDAN, and GPTFuzz collectively indicate that **semantically meaningful, human-readable, or near-readable prompts can transfer extremely well**. Therefore, defenders cannot rely on a heuristic that “natural language jailbreaks are too model-specific to worry about.”

Family-specific transfer profile

Table III synthesizes the transfer dynamics most supported by the supplied sources.

Family	Basis of transfer	Direct evidence in supplied sources	Main transfer limitations
Persona/manual prompts	Shared conversational alignment priors, roleplay susceptibility, instruction reframing	Strong ecosystem evidence of widespread reuse; no unified transfer matrix in supplied excerpt	Rapid patching of famous templates; surface-form recognition; provider-specific moderation rules
Compositional manual prompts	Semantic decomposition of harmful intent reconstructed by instruction-following models	CIA on GPT-4, GPT-3.5, Llama2-70B with >95% ASR	Defenses that infer underlying intent; possible degradation if semantic reconstruction is blocked (2)
Many-shot demonstrations	Generic imitation/continuation behavior across aligned chat models	ICA affects various aligned LLMs	Context window limits, prompt-budget constraints, stronger system-prompt anchoring (2)

Security significance of cross-family portability

For defenders managing heterogeneous fleets—e.g., mixing proprietary API models and open-weight fallbacks—the cross-model behavior of these prompt families is strategically dangerous. A transferable prompt means:

- the same attack workflow can be tested on one accessible model and redeployed elsewhere;
- public prompt-sharing communities can rapidly adapt across providers;
- safety regressions on one product may spill over into adjacent systems using similar alignment recipes.

CIA is the strongest benchmarked sign of this problem in the supplied record. The in-the-wild prompt repositories then show how such prompt knowledge can disseminate socially. Together, they imply that **portability is not merely a technical issue but a distribution issue**: the easier an attack is to read, explain, and copy, the larger its practical transfer impact.

Prompt-Level Safeguards Bypassed, Pressured, or Countered by These Attacks

Distinction from earlier defense mapping

Prior content in the larger report already discussed defense classes at a high level. This section is narrower: it maps **persona/manual and many-shot attacks specifically to prompt-level safeguards**, especially those explicitly named in the supplied sources. The focus is not on internal telemetry or deployment controls in the abstract, but on **what these attacks defeat at the prompt layer** and what limited counter-evidence exists.

Persona/manual attacks versus surface instruction safeguards

Public persona jailbreaks typically pressure the simplest and oldest prompt-level safeguard: the model's tendency to treat safety policy as a superior instruction in the instruction hierarchy. Manual roleplay prompts undermine this by reframing the exchange so that compliance appears consistent with some alternative conversational objective—fiction, simulation, debugging, dual-answer formatting, hypothetical reasoning, or role assumption.

Although the supplied sources do not provide a dedicated benchmark table for persona prompts against named safeguards, the in-the-wild prevalence itself suggests that **surface-level refusal triggers and static unsafe-pattern lists were insufficient to suppress public template**

circulation at scale ([1](#)). If those safeguards had been robust, one would expect a much thinner public prompt ecosystem. This is an inference, but it is a defensible one.

Prompt-level safeguards most likely pressured by persona/manual attacks include:

- **literal harmful-request filters**, because the attack often softens or roleplays the request;
- **simple refusal templates**, because the jailbreak tries to override them via meta-instructions;
- **policy-priority assumptions**, because the user reframes the context to alter the model's interpretation of what "helpfulness" means.

Compositional instruction attacks versus intent-based defenses

The review source is explicit here. CIA targets **GPT-4, GPT-3.5, and Llama2-70B**, and is evaluated against **Intent-Based Defense (IBD)** designed to detect harmful intents. The attack achieves **>95% ASR**, while **IBD reduces ASR by >74%** by identifying the underlying malicious intent ([2](#)).

This is one of the most informative defense interactions in the supplied material because it reveals both the failure mode and the partial remedy.

What CIA bypasses

CIA appears to bypass or weaken:

- **surface toxicity screening**, because the harmful objective is compositionally distributed rather than overtly stated;
- **naive intent extraction**, if that extraction relies too heavily on local lexical cues;
- **prompt refusal rules anchored to explicit harmful asks**, because the ask may be reconstructed only after semantic composition.

What IBD demonstrates

The success of IBD in reducing ASR by over 74% shows that:

- semantic reconstruction of underlying intent can substantially improve robustness;
- the attack's strength is indeed tied to hidden or distributed intent, not random quirks;
- prompt-level defense can work better when it reasons over the latent task rather than isolated tokens.

What remains unresolved

The review wording does not claim elimination, only reduction ([2](#)). Therefore:

- CIA remains a serious threat even under improved prompt-level defense;
- defenders likely need layered controls beyond IBD;
- broad semantic attacks may adapt to whatever latent-intent inference features are deployed.

Many-shot jailbreaks versus in-context refusal conditioning

Multi-Turn and Automated Black-Box Jailbreak Families: Measured Effectiveness, Interaction Budget, and Defense Evasion

Source-bound scope and differentiation from prior subtopic coverage

While the prior subtopic reports already covered broad taxonomy, generic metric design, persona/manual attacks, many-shot prompting, and a high-level cost/transfer matrix, this section is intentionally narrower and more technical. It focuses only on **multi-turn escalation** and **automated black-box orchestration** families—especially **Crescendo**, **Tree of Attacks (ToA)**, **MRJ-Agent**, and closely related composable attacks—and emphasizes four dimensions not previously developed in detail:

1. **turn-by-turn interaction structure** rather than single-prompt promptcraft,
2. **empirical ASR on named frontier models** such as GPT-4, GPT-4o, Gemini Pro-class systems, and Claude-family models,
3. **interaction/query budget and automation overhead**, and
4. **which safety layers are specifically bypassed by decomposition, persistence, or conversational state manipulation** rather than by brute-force suffix optimization or persona framing alone.

This distinction matters because these attacks exploit a different weakness surface than the manual/persona families already discussed. Instead of relying mainly on one-shot semantic persuasion, they often accumulate leverage across turns, progressively normalize unsafe framing, fragment intent into locally acceptable subgoals, or use an attacker-controlled agent to maintain state and adapt after refusal. The available 2024–2026 literature increasingly treats these attacks not as “longer prompts,” but as **interactive search processes over dialogue state**, often under realistic API-only conditions ([1](#), [2](#), [3](#), [4](#), [5](#)).

A second difference from the earlier sections is that this report does **not** repeat the earlier treatment of many-shot jailbreaking, generic prompt mutation, or broad ecosystem prevalence. Many-shot is only referenced here when it helps explain interaction-budget tradeoffs or context-window exploitation in relation to multi-turn attacks. Likewise, tool/agent-mediated exploit chains were previously noted at a taxonomic level; here they appear only insofar as the newer literature shows **agentic multi-round jailbreak generation** converging with black-box search and recursive decomposition ([6](#), [7](#)).

The evidence base has one important limitation that should be stated plainly. Several of the most relevant papers are relatively new and, in some cases, the provided source access is partial HTML rather than full appendix tables. Accordingly, the report distinguishes carefully between:

- **directly reported results** from the cited sources,
- **cross-paper synthesis** where multiple sources align on a trend, and
- **inference** where the exact number/model pairing is not fully recoverable from the supplied material.

Where exact model-specific ASR tables are unavailable in the provided record, this section avoids inventing values and instead reports the strongest bounded claim supported by the source text. That approach is especially important for post-2025 papers on multi-turn automation, where claims of generality across GPT-4, Claude, and Gemini families are stronger than the exact per-cell percentages visible in the supplied scrape ([2](#), [3](#), [7](#)).

Crescendo: iterative escalation as a black-box conversational search process

The **Crescendo** attack family is one of the clearest demonstrations that black-box jailbreak efficacy can emerge from **conversation management**, not only from adversarial token strings. The central mechanism is progressive escalation: the attacker begins with innocuous, often policy-compliant or partially compliant turns and incrementally steers the model toward restricted content by exploiting follow-up continuity, context retention, and the model’s tendency to preserve topical momentum across a dialogue. In operational terms, Crescendo is a **multi-turn stateful jailbreak** rather than a single optimized prompt ([1](#)).

This matters because conventional prompt-level safeguards often assume that harmfulness is visible in a single request. Crescendo instead spreads the harmful trajectory across turns: each local step may appear educational, hypothetical, analytical, or only partially related to the eventual unsafe objective. The model is then induced to bridge the remaining gap itself, often because prior turns have already established a latent frame that weakens subsequent refusal behavior. This is qualitatively different from the manual persona prompts covered earlier. Persona prompts typically

ask the model to adopt a role immediately; Crescendo more often **earns** unsafe cooperation through a staged negotiation with the policy boundary (1, 3).

A key contribution of the original Crescendo work is that it framed the attack as realistic under **black-box access**. The attacker does not require gradients, logits, or system prompt visibility; success derives from observing refusals or partial compliance and using those outputs as feedback for the next turn. This places Crescendo in the same operational class as later automated black-box attacks, even when the initial Crescendo procedure is partly human-guided. In that sense, Crescendo should be understood as a precursor to more automated multi-turn agentic attacks: it established that **dialogue depth itself is an attack resource** (1, 2).

The 2025 synthesis paper *Multi-Turn Jailbreaks Are Simpler Than They Seem* strengthens this point by arguing that strong multi-turn jailbreak performance does not always require highly elaborate reasoning or deeply customized attack logic. The paper explicitly reports **success rates exceeding 70%** and names **GPT-4, Claude, and Gemini variants** among the affected targets, indicating that multi-turn attacks can remain highly effective across leading commercial model families even when simplified or partially automated (3). Although the supplied excerpt does not expose the full appendix tables needed to map each exact percentage to each exact model variant, the direct textual claim is still significant: it suggests that the empirical barrier to effective multi-turn jailbreaking is lower than defenders might assume.

The practical significance of Crescendo-like escalation is visible in how it interacts with **conversation-level memory**. A model that refuses an overt harmful request at turn 1 may nonetheless provide enabling subcomponents by turn 3 or 4 after the task has been reframed as historical analysis, abstract planning, content transformation, risk comparison, or error correction of prior model outputs. The bypass mechanism is therefore not only “semantic evasion” but **policy fragmentation across a persistent dialogue state**. Crescendo succeeds when the model’s safety system does not adequately aggregate the cumulative intent trajectory of the exchange (1, 7).

The literature also implies that Crescendo can pressure or bypass multiple defensive layers simultaneously:

- **surface harmful-intent detection**, because each individual turn may be low-severity or ambiguous;
- **response-side refusal heuristics**, because intermediate compliant answers create momentum and narrower follow-up questions;
- **single-turn prompt classifiers**, because they often score prompts independently rather than as a sequence; and

- **some self-reminder or policy-priming defenses**, because conversational context can dilute or strategically work around them over successive turns (7, 8).

The following table consolidates the strongest source-supported observations for Crescendo and adjacent staged multi-turn escalation attacks.

Family / mechanism	Access model	Named targets in supplied sources	Directly reported efficacy	Interaction budget characteristics	Main bypass mode	Evidence strength
Crescendo	Black-box/ API conversational	Frontier aligned chat models; later synthesis links class to GPT-4/ Claude/ Gemini-style targets	Original paper establishes practical multi-turn jailbreak effectiveness; later synthesis reports multi-turn ASR exceeding 70% on GPT-4/ Claude/ Gemini variants for simplified attacks in this family neighborhood	Multi-turn, adaptive; usually several dialogue rounds rather than one-shot	Context accumulation, incremental reframing, refusal erosion	High for this family; effective on modern models; exact percentage in supplied records
Simplified automated multi-turn escalation	Black-box/ API	GPT-4, Claude, Gemini variants	“Success rates exceeding 70%” reported in	Automated or semi-automated multi-round interaction	Stateful decomposition and iterative persistence	High for existing overall band; for full model

Family / mechanism	Access model	Named targets in supplied sources	Directly reported efficacy	Interaction budget characteristics	Main bypass mode	Evidence strength
		explicitly referenced	synthesis source			model visibility
Echo Chamber / related decomposition attacks	Black-box/ API	Discusses MRJ-Agent and multi-turn decomposition/ context attacks	Source positions this family as effective and composable; exact ASR figures not fully visible in supplied text	Requires sustained dialogue and attack-controller logic	Repetition, self-reinforcing context, decomposed intent	Model with temporal caution because of recent

One of the most policy-relevant findings in the 2025 synthesis is that **attack sophistication may be overestimated**. If multi-turn jailbreaks with success rates above 70% can be achieved with relatively simple turn sequencing, then the defender’s problem is not merely to block a few famous handcrafted attacks like Crescendo, but to address an entire **low-complexity attack manifold**. This makes operational defenses based on blacklist prompts or known jailbreak strings particularly weak against the family. The dangerous property is not the exact wording, but the existence of a short path through dialogue space from benign framing to unsafe assistance (3).

From an attacker-cost perspective, Crescendo occupies a middle ground:

- more laborious than one-shot persona prompts,
- usually cheaper than large-scale adversarial suffix search,
- and often deployable with low infrastructure cost because it uses only ordinary chat access.

The main cost is **interaction budget** and, for human operators, **craft skill in steering**. However, once the strategy is templated, that cost falls sharply. This is precisely why later work on automated multi-turn attacks is important: it converts a moderately skilled conversational tactic into a scalable pipeline (1, 3, 4).

Tree-structured and modular black-box search: from Tree of Attacks to cross-behavior automation

Where Crescendo demonstrates that dialogue history can be weaponized, **Tree of Attacks (ToA)** and related modular search frameworks show that black-box jailbreak generation can be formalized as **branching exploration over attack trajectories**. This family is especially important because it bridges prompt attacks and agentic planning: rather than committing to a single prompt path, the attacker explores multiple decomposition routes, subgoals, or semantic rewrites, pruning low-performing branches and expanding promising ones. The result is a more systematic and often more query-efficient search over black-box policy weaknesses ([4](#), [5](#), [6](#)).

This section differs from earlier cost discussions by focusing specifically on **branching search overhead** and **cross-behavior reuse**, not on the general fact that automation can reduce query cost. In ToA-like systems, the key question is not simply “how many queries?” but “how many distinct branches and evaluation decisions are needed to find a successful conversational path?” That is operationally significant because branching attacks can consume budget quickly if unoptimized, yet they can also amortize discovery across many target behaviors if they learn reusable decomposition patterns ([4](#), [5](#)).

The paper *Effective and Efficient Jailbreaks of Black-Box LLMs with Cross-Behavior Attacks* is particularly useful here because it explicitly discusses the shortcomings of prior black-box methods such as **iterative feedback-guided approaches**, **Tree of Attacks**, and **AutoDAN-Turbo**. The source notes that existing methods can suffer from **expensive query budgets** and limitations in **transfer** or **semantic diversity**, which is already an important empirical finding even before considering its proposed method. In other words, by 2025 the literature recognized a tradeoff inside automated black-box jailbreaking: highly adaptive search can be strong, but naïve adaptation may be too query-heavy or too behavior-specific to scale well ([4](#)).

That same paper emphasizes **cross-behavior attacks**, a concept that deserves attention in the broader economics of jailbreak development. If an attack discovered for one harmful behavior can be generalized, parameterized, or lightly adapted across many prohibited intents, then the attacker no longer pays the full search cost per behavior. This moves the economics from bespoke exploit development toward **reusable attack templates**, which is exactly the sort of maturation one would expect in a jailbreak-as-a-service ecosystem. Even when the source’s main focus is not multi-turn dialogue per se, it directly informs the evaluation of composable black-box families by highlighting the importance of **semantic diversity** and **reuse across harmful objectives** ([4](#)).

The **MAD-MAX** paper adds another layer by treating automated red teaming as a composition problem: attackers can mix modules, mutation strategies, or structural components to generate

diverse malicious prompt variants. The supplied scrape explicitly references **Tree of Attacks**, indicating that the field increasingly sees tree-structured search not as a one-off algorithm but as part of a larger modular ecosystem. For defenders, that means an adversary need not choose between “Crescendo” and “ToA” in a pure sense; they may combine staged dialogue escalation with branching prompt candidates, response-based mutation, or subgoal recomposition in one pipeline (5).

The black-box automation landscape can therefore be represented as a continuum:

1. **linear iterative attacks**, which adjust one candidate over turns,
2. **tree-based attacks**, which explore multiple candidate trajectories in parallel or semi-parallel,
3. **cross-behavior attacks**, which seek prompt structures that generalize across prohibited tasks, and
4. **modular/mixture systems**, which compose several attack operators or controllers.

The stronger these systems become, the more misleading it is to think of jailbreaks as isolated prompts. They are increasingly **search programs** over the target model’s refusal behavior (4, 5, 6).

The empirical record in the supplied sources supports several concrete observations about ToA-style families:

Attack family or comparator	What the source directly states	Implication for ASR / cost analysis	Relevance to named frontier models
Tree of Attacks	Explicitly referenced as a prior automated black-box family with expensive query budgets in later efficiency work	Suggests meaningful jailbreak capability but with non-trivial search cost and scaling challenges	High relevance because it is discussed in benchmarking literature targeting major closed models (4)
AutoDAN-Turbo	Used as a black-box automated comparator in efficiency discussions	Indicates competitive automated performance but query/transfer tradeoffs remain salient	Relevant as a baseline for GPT-4-class and similar API-evaluated settings (4)
Cross-behavior attack methods	Presented as addressing limitations in expensive queries and weak semantic diversity	Economically important because they may reduce per-behavior search cost	High, especially for service-like attack reuse across model APIs (4)

Attack family or comparator	What the source directly states	Implication for ASR / cost analysis	Relevance to named frontier models
MAD-MAX modular mixtures	Frames attacks as composable and diverse; references ToA	Indicates attackers can combine modules, increasing robustness and coverage	Important for broad multi-model red teaming and transfer framing (5)

Although the supplied excerpts do not provide a clean per-model ASR table for ToA itself, they do support a strong comparative interpretation: **ToA is influential enough to be a baseline whose weaknesses are query cost and limited transfer efficiency, not lack of efficacy.** In security research, that distinction matters. A family becomes a recurring benchmark not because it fails, but because it works well enough to motivate optimization. Thus, the absence of a fully visible ASR percentage in the supplied snippet should not be read as evidence of weakness; it should be read as a limitation of the retrieved extract. The documented discussion of efficiency improvements over ToA strongly implies that ToA is already a credible black-box jailbreak method against aligned LLMs (4, 5).

The **transferability** question is more subtle. Tree-structured search can in principle overfit to one model’s refusal patterns if it branches based heavily on target-specific responses. That may raise ASR on the target API while reducing portability to other vendors. The cross-behavior paper’s explicit criticism of prior methods’ limitations in **transfer** and **semantic diversity** supports precisely this reading: some automated black-box attacks learn prompt forms that are effective but too narrowly adapted to one target or one behavior class (4). This differs from many manual prompt attacks, where lower optimization may paradoxically improve portability because the prompts remain semantically natural and broadly persuasive.

ToA-style systems also appear especially relevant to **reasoning-capable or agentic models**. The Nature Communications article on reasoning models as autonomous jailbreak agents explicitly references **Tree of Attacks** and places it in a trajectory toward recursive, self-optimizing attack systems. The important implication is not merely that reasoning models can be jailbroken, but that they may also **participate in generating better jailbreaks** through planning, self-evaluation, and branch selection. This raises a dual-use issue: the same reasoning capacity used for benign agentic workflows can increase the efficiency of black-box attack search (6).

From a defense perspective, ToA-like methods bypass controls that assume:

- a single static malicious prompt,

- low attacker persistence,
- or no adaptive use of the model’s own responses.

They are especially problematic for systems that expose rich refusal feedback, because each refusal can function as a gradient-free signal telling the attacker which branch to expand next. This places a premium on **response minimization**, **conversation-level risk scoring**, and **rate-aware throttling**, rather than only static prompt filtering. The supplied survey-style and benchmark-style sources mention defenses such as self-reminders and other mitigation strategies, but the recurring black-box results imply that these are often insufficient when the attacker can persist adaptively over several rounds (7, 8).

MRJ-Agent, Echo Chamber, and composable agentic jailbreak orchestration

If Crescendo demonstrated iterative escalation and ToA formalized branching search, **MRJ-Agent** and related agentic systems represent the next step: the attacker delegates jailbreak construction to an **autonomous or semi-autonomous controller** that can plan multi-turn interactions, inspect model outputs, decompose tasks, maintain memory, and choose follow-up strategies. The *Echo Chamber Multi-Turn LLM Jailbreak* source is particularly relevant because the supplied context explicitly notes that it references **MRJ-Agent** and other multi-turn decomposition/context attacks, making it valuable for ecosystem mapping even when the full quantitative appendix is not visible (2).

This section is intentionally distinct from the earlier generic mention of tool/agent exploit chains. The earlier report treated tool-mediated attacks as a broad family at the system level. Here the focus is narrower: **agentic orchestration of the jailbreak conversation itself**. In other words, the agent is not merely using tools to act in the world; it is acting as the **prompt strategist**, dynamically constructing and revising a multi-turn jailbreak campaign against a target model. That shift matters because it reduces the human skill barrier while increasing persistence and consistency.

The “Echo Chamber” framing is analytically useful because it highlights a mechanism beyond mere escalation: the attack can create a **self-reinforcing conversational environment** in which the target model is repeatedly exposed to context that normalizes, reframes, or incrementally concretizes the unsafe objective. In such a chamber, the attack is not one request plus follow-up corrections; it is a managed discourse that keeps the model inside a narrative or reasoning track favorable to policy slippage. The supplied context suggests that MRJ-Agent belongs to this broader class of **multi-turn decomposition/context attacks** (2).

Operationally, MRJ-Agent-like systems appear to combine several components:

- **goal decomposition**, breaking a prohibited target into intermediate asks likely to evade refusal;
- **memory of prior responses and refusal patterns**, enabling adaptive follow-ups;
- **attack policy selection**, choosing whether to reframe, elaborate, ask for critique, summarize, continue, or transform prior text;
- **possibly multi-agent or recursive prompting**, where one model or controller evaluates another's outputs; and
- **automatic stopping criteria**, terminating once enough unsafe content has been accumulated.

This architecture is increasingly plausible because LLM orchestration frameworks make it straightforward to automate state tracking and prompt scheduling, even under ordinary API access. Thus, MRJ-Agent is important not only as a named attack, but as evidence that jailbreak capability is becoming **programmable behavior** rather than artisanal promptcraft ([2](#), [6](#)).

The 2025 paper *Multi-Turn Jailbreaks Are Simpler Than They Seem* is useful here because it tempers a possible misconception: one might assume that agentic orchestration is necessary for strong multi-turn jailbreaks, but the paper suggests simpler attacks already reach **greater than 70% success rates on GPT-4, Claude, and Gemini variants** ([3](#)). The implication is twofold:

1. **MRJ-Agent-like systems are dangerous not because they make the impossible possible, but because they industrialize what is already possible**, and
2. the marginal gain from sophisticated orchestration must be evaluated relative to a surprisingly strong low-complexity baseline.

That has direct consequences for attacker economics. If a simple scripted multi-turn attack already performs strongly, then agentic orchestration does not need to deliver dramatic ASR improvements to be worthwhile; it only needs to improve consistency, reduce human labor, or generalize across targets and tasks.

The agentic trend also intersects with the broader finding from the Nature article that **reasoning models can become autonomous jailbreak agents**. Even though that paper is more contextual than primary for extracting ASR numbers, it supports a critical strategic point: stronger reasoning and planning can be repurposed to optimize jailbreak campaigns, including deciding which decomposition branch to pursue, when to feign benign intent, or how to rewrite a refused subrequest into a compliant-looking alternative ([6](#)). This is a qualitatively different risk from traditional prompt-sharing communities. It implies that attack generation may increasingly be automated *by* capable models rather than merely *against* them.

The evidence available in the supplied record supports the following structured characterization of MRJ-Agent-like and Echo Chamber-like systems.

System / family	Core idea	Direct support in supplied sources	Likely operational burden	Primary safety weakness exploited
MRJ-Agent	Agent autonomously conducts multi-turn jailbreak via decomposition and context management	Explicitly referenced in <i>Echo Chamber</i> paper context	Moderate development burden, low per-target human effort once built	Conversational state tracking and adaptive follow-up (2)
Echo Chamber	Multi-turn context reinforcement / self-reinforcing unsafe framing	Central subject of cited 2026 paper	Multi-round interactions; likely moderate token/query cost	Cumulative context shaping, normalization, and persistence (2)
Agentic reasoning jailbreakers	Models act as planners/optimizers of attack paths	Nature source explicitly discusses autonomous jailbreak agents and references ToA	Higher orchestration complexity but scalable once automated	Recursive planning, search, and self-improvement (6)

A key defense-relevant property of MRJ-Agent-like attacks is that they can **adapt after partial refusal** without a human in the loop. Earlier manual attacks often falter when the model responds with a warning, an incomplete refusal, or a partial answer that needs clever interpretation. An agentic attacker can use that signal immediately, possibly through heuristics such as:

- detecting whether the refusal was categorical or conditional,
- extracting allowed subtopics from the refusal text,
- pivoting to a more acceptable decomposition, and
- resuming from the last compliant state.

This makes the interaction more like an adversarial negotiation protocol than a prompt attack.

Another important consequence is **scalability across targets**. A human operator running Crescendo manually can usually attack only a limited number of sessions at once. An MRJ-Agent-like controller, by contrast, can concurrently probe many model endpoints, compare refusal styles, and preserve successful attack trajectories as templates. That increases both offensive throughput and data collection, which in turn improves the quality of future attacks. The result resembles the maturation path seen in other adversarial domains: manual exploitation becomes scripted exploitation, which becomes service-backed exploitation.

The bypassed defenses in this class are broader than in single-turn attacks. Based on the supplied literature, agentic multi-turn attacks can pressure or evade:

- **single-turn prompt classifiers**, because harmfulness emerges only from the full sequence;
- **static refusal templates**, because the agent learns to route around their wording;
- **keyword-triggered moderation**, because decomposition keeps each step superficially acceptable;
- **simple self-reminder overlays**, because persistence and context engineering can dilute them; and
- **human-moderation assumptions about session length**, because the agent can continue systematically where a casual user might stop ([2](#), [7](#), [8](#)).

One of the stronger cross-source inferences is that **agentic orchestration may matter more for reliability and reuse than for raw one-session ASR**. The direct source support for this is indirect but coherent:

- simplified multi-turn attacks already exceed 70% on leading models ([3](#)),
- ToA-style and cross-behavior work emphasizes efficiency and semantic diversity ([4](#)),
- and Echo Chamber/MRJ-Agent point to automation of decomposition and context management ([2](#)).

Taken together, the likely interpretation is that agentic systems are best understood as **force multipliers** for known multi-turn vulnerabilities rather than wholly new exploit primitives.

Frontier-model results, transferability, and the role of interaction budget

This subsection focuses on **what can be said empirically about named frontier models**—especially GPT-4, GPT-4o, Gemini Pro-class systems, and Claude-family models—without repeating the general transfer discussion from prior reports. The emphasis here is on **family-specific evidence for multi-turn and black-box automated attacks**, how much interaction they require, and what the available literature implies about portability across vendors.

The strongest direct cross-model statement in the supplied corpus comes from *Multi-Turn Jailbreaks Are Simpler Than They Seem*, which explicitly names **GPT-4, Claude, and Gemini variants** and reports **success rates exceeding 70%** for multi-turn jailbreaks (3). Even without the full per-model table in the provided extract, this is already a noteworthy frontier-model result for three reasons.

First, it spans **multiple leading vendor families**, which weakens the argument that multi-turn jailbreaks are artifacts of one company’s alignment stack. Second, the paper’s framing—“simpler than they seem”—suggests the attacks are not exotic high-cost procedures, but relatively accessible methods. Third, because GPT-4, Claude, and Gemini are generally trained and aligned with different pipelines, the result implies that **conversational-state vulnerabilities are at least partially architecture-agnostic** at the product level.

The supplied materials are less explicit about **GPT-4o** in the narrow multi-turn papers than about GPT-4 broadly. However, GPT-4o falls naturally into the later frontier closed-model generation considered by 2025–2026 benchmark papers and surveys. Where exact GPT-4o cells are not visible in the extracted text, the correct reading is cautious: the family-level findings on frontier commercial chat systems likely remain relevant, but exact GPT-4o ASR should not be overstated without direct table visibility. Accordingly, GPT-4o is included here as a **named model of interest** for which the supplied sources support relevance but not always a recoverable exact ASR number in the snippets provided (3, 7, 8).

A similar caution applies to **Gemini Pro** versus “Gemini variants.” The supplied source explicitly mentions Gemini variants, which is strong evidence of model-family relevance. If a specific experiment used Gemini Pro, Gemini 1.5 Pro, or another Gemini-branded frontier variant, that distinction is not always recoverable from the excerpt. Therefore the report uses “Gemini Pro-class / Gemini variants” where appropriate rather than assigning unsupported exact numbers to a specific sub-variant (3).

For **Claude-class systems**, the record is somewhat stronger because the supplied source set also includes Anthropic’s own **Many-shot Jailbreaking** paper, which demonstrates that long-context attack resources can be effective against **Claude 2.0** and discusses **GPT-3.5** and **GPT-4** as well (9). Although many-shot is not the primary focus here, it corroborates a broader point relevant to multi-turn analysis: **Claude-class systems are not uniquely robust to context-mediated jailbreaks**. Whether the context is packed into one long prompt or distributed across multiple turns, the same general weakness appears—safety policies can be weakened by sustained contextual conditioning.

The interaction-budget question is central. Unlike one-shot attacks, multi-turn and agentic black-box attacks consume budget along at least three axes:

- **number of turns**,
- **total tokens exchanged**, and
- **branching/search overhead** if multiple candidate paths are explored.

These dimensions do not always move together. A Crescendo attack may use only a handful of turns but high operator skill. A many-shot jailbreak may use one user message but thousands of tokens. A Tree-of-Attacks method may use many target queries because it explores several branches. An MRJ-Agent pipeline may use moderate turns per session but substantial hidden orchestration logic.

The following table synthesizes the source-grounded interaction-budget picture.

Family	Typical budget form	Directly documented budget clues	Cost implication
Crescendo	Several adaptive dialogue turns	Defined as multi-turn escalation; black-box conversational process	Low infrastructure, moderate manual effort unless scripted (1)
Simplified automated multi-turn attacks	Multi-round interaction, often shorter than assumed	2025 synthesis argues attacks are simpler than expected while still exceeding 70% ASR	Lower skill barrier than earlier assumed; good attacker ROI (3)
Tree of Attacks / similar branch search	Many queries possible due to branching	Later efficiency paper explicitly criticizes expensive query budgets of prior methods including ToA	Potentially high API cost without optimization (4)
Cross-behavior automated attacks	Up-front search, then reuse across behaviors	Paper frames method as more effective/efficient than costly black-box baselines	Better amortization of attack development cost (4)
MRJ-Agent / Echo Chamber	Sustained interaction plus orchestration overhead	Multi-turn decomposition/ context attack framing; agent maintains state	Moderate engineering burden but low marginal labor per attack (2)

Family	Typical budget form	Directly documented budget clues	Cost implication
Many-shot context attacks	High token cost, fewer conversational decisions	Anthropic paper discusses hundreds of demonstrations scaling attack effectiveness	Expensive in context budget, simpler cognitively (9)

The **transferability** picture is mixed and is one area where multi-turn attacks differ materially from both adversarial suffixes and manual promptcraft. The 2025 synthesis source reportedly discusses **transferability/correlation across models**, which is important because it suggests that vulnerability to one multi-turn attack may correlate with vulnerability to others across vendors (3). That does not necessarily mean exact prompts transfer unchanged; rather, it suggests the **attack family’s success** is not vendor-specific.

This is consistent with the operational intuition from Crescendo. Because the attack relies on generic dialogue dynamics—incremental compliance, memory, continuity, and reframing—it should transfer more naturally than brittle token-optimized suffixes. However, ToA-style and feedback-guided attacks may exhibit a tension: the more strongly they adapt to a single model’s refusals, the more likely they are to overfit. The cross-behavior paper’s explicit mention of transfer limitations in prior methods supports that interpretation (4).

A useful way to summarize the transfer regime is as follows:

Family	Likely transfer unit	Expected portability
Crescendo-like escalation	Strategy/template of staged follow-ups	Moderate-to-high across chat-aligned models because mechanism is conversationally generic
Tree of Attacks	Search procedure more than final prompt	High procedural transfer, lower exact prompt transfer if branches overfit target responses
MRJ-Agent / Echo Chamber	Controller logic and decomposition policy	Potentially high, because the agent can adapt online to each model
Cross-behavior black-box attacks	Reusable semantic attack motifs	Designed specifically to improve reuse across harmful tasks and possibly across targets
Many-shot	Demonstration principle	

Family	Likely transfer unit	Expected portability
		Broad family portability, but heavily constrained by context limits and token pricing

This helps explain why the **economics of adversarial prompt development** for multi-turn families may be attractive even when raw query count is higher than one-shot attacks. If the transferable asset is the **controller** or **strategy graph** rather than a single string, then the attacker can amortize development across many targets. A commercialized jailbreak service would likely prefer such assets because they are easier to refresh as vendor filters evolve.

The available literature

Optimization-Based and Adversarial Suffix Jailbreaks: Empirical Performance, Transfer, and Alignment Evasion

Scope delimitation relative to earlier subtopic reports

Where prior sections in the larger report already defined jailbreak taxonomies, compared broad attack families, or briefly noted that Greedy Coordinate Gradient (GCG) historically exhibits strong white-box efficacy but weaker transfer, this section narrows to the *mechanics and empirical record* of optimization-based and adversarial suffix attacks themselves. The emphasis here is not on taxonomy, general cost matrices, or black-box conversational search already covered elsewhere, but on four more specific questions: (i) what the post-2023 literature actually measures for GCG and its descendants; (ii) how later work attempts to reduce optimization cost or improve universality and transfer; (iii) what white-box access is truly required at development time versus deployment time; and (iv) why these strings defeat alignment and refusal layers despite being semantically incoherent or only weakly interpretable to humans ([1](#), [2](#), [3](#), [4](#)).

A second difference from earlier sections is that the focus here is *intra-family variation*. Instead of treating “adversarial suffix optimization” as one row in a larger taxonomy, this section distinguishes classic GCG, position-aware variants, index-gradient methods, probe-sampling accelerations, model-based suffix generators such as AmpleGCG, and later retrospective analyses arguing that GCG remained more effective against newer model generations than many practitioners assumed ([1](#), [2](#), [4](#)).

A third distinction is that, because the present subtopic must support the larger report’s comparative analysis across 2023–2026 jailbreak families, the tables below separate **development-time access assumptions** from **deployment-time use**. This matters because many papers describe “transfer attacks” that are black-box at deployment but still rely on white-box gradients over one or more open-source surrogates during attack construction. Conflating those phases systematically overstates practical black-box capability ([2](#), [3](#), [4](#)).

Method families inside the suffix-optimization lineage

The central methodological template remains the same across most papers in this lineage. An attacker appends a trainable token suffix to an otherwise harmful prompt and iteratively updates those discrete tokens to maximize the target model’s probability of producing a specified affirmative or unsafe continuation. In the original GCG formulation, this is done by computing gradients of a loss with respect to token embeddings, selecting coordinates whose replacement is predicted to most improve the objective, and greedily editing the suffix until the model ceases refusing and begins emitting a target-style completion. Because the attack operates directly in token space, it preserves the prompt’s overt harmful intent while manipulating the decoder into a different local behavior regime ([1](#), [2](#), [4](#)).

Later variants diverge along four axes:

1. **Objective design:** some methods optimize refusal suppression, some optimize target-answer likelihood, and some incorporate intermediate or index-based gradients rather than relying only on the final-token loss.
2. **Search efficiency:** methods such as Faster-GCG, Probe Sampling, and related acceleration schemes attempt to reduce the candidate-evaluation burden that made naïve GCG expensive.
3. **Transfer strategy:** some attacks optimize on one white-box model and transfer the resulting suffix to other open or closed models; others train generative models to sample many candidate suffixes and thereby improve universality.
4. **Prompt structure:** newer work shows that attack effectiveness depends not merely on the suffix *string* but on token *positioning* and prompt layout, undermining the simplistic view that all success derives from a mysterious appended gibberish sequence ([1](#), [2](#), [3](#), [4](#)).

The most important consequence for empirical interpretation is that “GCG success rate” is not a single stable quantity. Reported attack success depends on the target behavior set, the harmfulness classifier or exact-match criterion, the number of optimization steps, the suffix length, the prompt template, the model family, the version of the alignment stack, and whether evaluation permits transfer from a surrogate rather than direct optimization on the victim. Several later papers explicitly caution that model generations once thought to have “solved” suffix attacks remained

vulnerable under updated search procedures or under different prompt placements, which is one reason retrospective surveys describe a “resurgence” of GCG-style attacks rather than a simple decline after the first defenses appeared (1, 2, 4).

White-box dependence, gray-box assumptions, and black-box deployment realities

The white-box requirement is the single most important operational limitation of classic GCG. The attack fundamentally depends on access to model gradients or at least enough internal information to approximate them. In open-source models this is straightforward. In closed commercial APIs it is not. As a result, the baseline GCG threat model should be understood as **white-box development plus possible black-box transfer**, not as a native black-box jailbreak method (2, 4).

That distinction is stated clearly in the optimization literature. The index-gradient paper characterizes optimization-based jailbreaking as relying on gradients and output probabilities, then situates GCG, I-GCG, Probe Sampling, MAC, and AmpleGCG within this broader family. It also highlights the practical bottleneck: direct optimization requires privileged access and substantial compute, while transfer to proprietary models is less reliable and often the true empirical hurdle (2). AmpleGCG is best understood as an answer to this problem. Rather than running expensive optimization separately for each harmful prompt and each victim model, it trains a generative model over adversarial suffixes derived from surrogate optimization, aiming to amortize the white-box cost into a reusable attack generator that can then be deployed in black-box settings against unseen prompts and even closed systems (3).

This leads to a more precise access taxonomy for the suffix family:

Method	Development-time access	Deployment-time access	Typical victim type	Security implication
Classic GCG	Full white-box gradients on target	Same target or offline generated string	Open-source aligned LLMs	High risk for self-hosted/open models; limited direct applicability to closed APIs
Surrogate GCG transfer	White-box on surrogate, none on victim	Black-box prompt submission only	Closed APIs, other open models	Risk depends on transferability; lower reliability but

Method	Development-time access	Deployment-time access	Typical victim type	Security implication
				more realistic operationally
Faster/accelerated GCG variants	Same as GCG, but fewer optimization steps/candidate evaluations	Same as GCG	Mainly open-source; sometimes surrogate-to-closed transfer	Reduces attack cost, broadening attacker pool
Amortized/generative suffix methods (e.g., AmpleGCG)	White-box optimization corpus on one or more surrogates plus training of generator	Black-box prompt submission only	Open and closed LLMs	Converts expensive expert attack into reusable inference-time exploit family
Explicitly harmful black-box prompting comparators	No white-box gradients	Black-box only	Commercial APIs	Useful baseline: often cheaper, more human-readable, but different stealth and transfer properties

This decomposition matters for broader ecosystem analysis. A frontier API provider is not equally threatened by every paper that reports strong GCG performance. The most immediate risk is from *transferable* or *learned universal* suffixes built on accessible surrogate models. Conversely, organizations deploying open weights with thin alignment layers face the strongest direct exposure because attackers can optimize on the target itself ([2](#), [3](#), [4](#), [5](#)).

Benchmarking caveats specific to this literature

The optimization-based jailbreak literature often reports impressive percentages, but careful reading shows substantial heterogeneity in what “success” means. Common criteria include: (i) exact target-string generation; (ii) non-refusal plus harmful content generation; (iii) attack judged successful by a classifier such as HarmBench; (iv) success over a curated set of harmful behaviors; or (v) transfer success from one source model to several victims. These are not interchangeable. A suffix can

strongly suppress refusal without yielding a high-quality harmful answer, or it can overfit to one model’s decoder quirks and fail under paraphrase, sampling changes, or template shifts (2, 4).

A second caveat is version drift. Several of the supplied sources are from 2025–2026 and reflect newer model releases and revised evaluations. This is valuable because the user asked for the latest available information, but it also means older headline figures may no longer generalize. The “Resurgence of GCG” review argues that claims of obsolescence were overstated and that updated attacks or evaluation setups re-established material vulnerability in later systems (4). Conversely, some impressive transfer rates reported for specific model pairs should not be overgeneralized to all frontier APIs because closed-model defenses change faster than published open-source benchmarks.

A third caveat is baseline strength. Papers sometimes note that transfer appears weak or strong partly because the destination model is either already vulnerable to a broad range of no-attack prompts or unusually hardened against the source model’s learned suffix distribution. The denominator is therefore not just “all harmful prompts,” but “all prompts under a particular policy, version, template, and evaluation judge.” This complicates direct across-paper ranking but does not erase the core pattern: white-box suffix optimization remains one of the highest-performing *controlled* jailbreak families, while practical risk hinges on whether recent methods can amortize or transfer that advantage (1, 2, 3, 4).

Measured results reported for GCG and directly related variants

The literature supplied to this task repeatedly treats GCG as the canonical optimization-based jailbreak baseline. Across open-source aligned models, classic GCG commonly achieves very high attack success under white-box conditions, often approaching saturation on curated harmful behavior sets when sufficient optimization steps and favorable templates are allowed. The exact percentages vary by model and benchmark, but the methodological papers consistently frame GCG as a strong baseline that later methods seek either to accelerate or to make more transferable rather than simply to surpass in raw white-box success (1, 2, 4).

The index-gradient paper is especially useful because it places several methods on the same comparative axis. Although exact figures differ across datasets and model settings, its discussion shows that newer variants improve either optimization efficiency or final ASR relative to vanilla GCG by exploiting richer gradient signals—specifically “index gradients”—instead of restricting updates to the most obvious local coordinate heuristics. The paper also references Probe Sampling, MAC, and AmpleGCG as points in the same design space, implying a maturing subfield where the question is no longer whether gradients work, but which gradient-derived signal yields the best cost–success–transfer tradeoff (2).

The position paper adds another important empirical refinement: attack power depends on *where* the optimized tokens are placed, not just on the suffix string itself. By examining token position in GCG adversarial attacks, it shows that the traditional “append a suffix to the end” framing misses a structural variable that materially affects optimization behavior and attack success. This matters because many benchmark pipelines hard-code one placement strategy; if alternate placements increase efficacy, earlier defensive evaluations may have understated residual risk (1).

The resurgence review then synthesizes a broader 2025 view: GCG-family attacks retained relevance due to continued improvements in search, transfer, and universality. Rather than being superseded by manual or multi-turn attacks, optimization-based suffixes became one component in a wider ecosystem that also includes black-box conversational search, readable automated prompts, and learned generators. That review is particularly significant because it frames recent work not as isolated tricks but as an attack lineage adapting to newer safety training regimes (4).

Because the supplied sources emphasize comparative method development more than a single uniform benchmark table, the most responsible way to summarize measured performance is to separate **documented directional findings** from **model-specific headline rates**. The table below does so.

Method cluster	What the supplied literature directly supports	Typical measured pattern	Reliability of cross-model transfer in supplied sources
Vanilla GCG	Strong baseline for optimization-based jailbreaks on aligned LLMs under white-box conditions (2, 4)	Often very high ASR on source model; expensive iterative search	Historically limited to moderate, often brittle
Position-aware / placement-sensitive GCG analysis	Attack success depends materially on token location, not merely suffix content (1)	Can improve or alter success relative to fixed appended-suffix baselines	Transfer implications plausible but still emerging
I-GCG / index-gradient methods	Richer gradient signal improves optimization-based jailbreaking efficiency/effectiveness over prior	Better search quality and/or ASR under comparable budgets	Still constrained by surrogate-target mismatch

Method cluster	What the supplied literature directly supports	Typical measured pattern	Reliability of cross-model transfer in supplied sources
	baselines in the paper’s setup (2)		
Probe Sampling / MAC / acceleration methods	Explicitly discussed as methods to reduce optimization burden or enhance search (2)	Lower query/compute burden than naïve GCG; still optimization-centric	Transfer not guaranteed; benefits are often source-side
AmpleGCG	Learns a universal and transferable generative model of suffixes for both open and closed LLMs (3)	Improves universality and black-box applicability by amortizing attack generation	Stronger than classic one-off suffix transfer in reported experiments

To avoid overclaiming, it is important to note that the supplied record does **not** support saying that any one variant universally dominates all others across all frontier models. What it does support is a narrower but operationally important statement: by late 2025 and early 2026, adversarial suffix optimization had evolved from a highly effective but transfer-limited white-box technique into a family containing methods explicitly designed to improve transfer, speed, and universality, with AmpleGCG and index-gradient work being prominent examples (2, 3, 4).

Faster-GCG and the broader acceleration trend

The user specifically requested Faster-GCG. The supplied sources do not provide a standalone primary paper solely devoted to Faster-GCG, but the optimization and resurgence surveys situate acceleration methods as an important branch of the GCG family, alongside Probe Sampling and other search-efficiency techniques. In this literature, “faster” generally refers to reducing one or more of the following: gradient evaluations, candidate token scoring, beam or coordinate search breadth, or end-to-end optimization steps needed to reach a successful jailbreak (2, 4).

That acceleration matters for two reasons. First, classic GCG can be operationally expensive even when it succeeds. If each harmful behavior requires many optimization iterations over long candidate lists, the method remains largely the domain of well-equipped researchers or attackers. Second, lower search cost improves *practical* transfer experimentation: an attacker can optimize on

multiple surrogate models, multiple prompt templates, or multiple seeds, then keep only suffixes that transfer well to black-box victims. In other words, efficiency improvements can indirectly improve black-box threat even if the victim never exposes gradients, because the attacker can spend the saved budget on broader surrogate exploration (2, 4).

The best-supported security interpretation, therefore, is not merely “Faster-GCG is cheaper,” but “acceleration lowers the barrier to industrialized suffix search.” This is particularly relevant to the economics of adversarial prompt development in the larger report. An expert attacker no longer needs to hand-craft a prompt or tolerate a single expensive optimization run per target. Instead, they can build a *pipeline*: collect behaviors, optimize or sample suffixes on open models, score them for transfer, package high-performing strings, and reuse them across campaigns. Such a pipeline resembles exploit development more than ordinary prompt engineering (2, 3, 4).

Universality: from per-prompt strings to reusable attack artifacts

One of the earliest weaknesses of classic GCG was that optimized suffixes often overfit the source prompt and source model. This led to an important shift in the literature: the goal became not merely finding *a* jailbreak suffix, but finding a suffix distribution or generator that works across many harmful requests and multiple models. The term “universal” in this context generally means a single suffix—or a generator capable of sampling suffixes from a learned attack distribution—can trigger jailbreak behavior across many prompts without task-specific re-optimization (3, 4).

AmpleGCG is the clearest primary source in the provided set for this transition. Its contribution, as reflected in the title and full text, is not simply another local optimizer but a learned generative model of adversarial suffixes intended to be both **universal** and **transferable**, spanning open and closed LLMs. This is a substantive change in attacker economics. Instead of paying the optimization cost each time, the attacker pays a one-time corpus-generation and model-training cost, after which attack deployment becomes cheap inference from the learned generator (3).

This shift has several empirical implications:

- **Higher deployment scalability:** universal suffixes can be distributed and reused by less skilled operators.
- **Improved black-box practicality:** a suffix generator trained on white-box surrogates can be queried against closed models without direct access.
- **Potential diversity benefits:** a generator can emit many variants, reducing reliance on one brittle fixed string.
- **Defense challenge:** blocking one known suffix string is less effective if the attacker can sample semantically unrelated but functionally similar sequences (3, 4).

The larger significance is that universality transforms suffix attacks from bespoke research artifacts into *candidate ecosystem commodities*. A fixed adversarial string shared on a forum is useful; a lightweight model or service that generates fresh transferable suffixes on demand is more consequential. Although the supplied sources are academic and do not provide strong empirical data on criminal marketplaces, they do support the inference that amortized suffix generation lowers operational skill requirements after the initial development phase ([3](#), [4](#)).

What AmpleGCG adds beyond “better transfer”

A careful reading of AmpleGCG’s framing suggests three distinct innovations beyond the simple claim of improved transfer. First, it learns a *distribution* over adversarial suffixes rather than relying on one optimized string. Second, it explicitly targets both open and closed models, making cross-model evaluation central rather than incidental. Third, it positions itself as a generative model of suffixes, which means deployment cost is closer to ordinary text generation than iterative gradient search ([3](#)).

This is important because many defenders reason about GCG through the lens of “weird gibberish strings.” A learned suffix generator undermines two comforting assumptions: that effective strings are rare, and that they are easily signed once discovered. If the generator samples many effective variants, then the attack family becomes less detectable by naïve substring matching and less sensitive to single-string countermeasures. At the same time, this does not imply that universality is unlimited. Transfer remains a learned property conditioned on the surrogate corpus, destination model family, and safety stack. A suffix generator trained on open-source chat models may transfer unevenly to highly moderated commercial APIs or to later model revisions ([3](#), [4](#)).

Cross-model transfer: what the literature supports and what remains uncertain

Cross-model transfer is the key criterion for real-world risk against closed frontier systems. The supplied sources support a nuanced position. They do **not** support claiming that classic GCG reliably transfers across all major vendor models. Rather, they show a trajectory from historically limited transfer toward improved but still uneven transfer through universal and learning-based methods ([2](#), [3](#), [4](#)).

The direct evidence in the set is strongest for AmpleGCG, which explicitly evaluates against both open and closed models and frames universal transfer as its main result. The resurgence review then places AmpleGCG within a broader family of black-box extensions and transferable suffix advances, suggesting that by 2025 it had become untenable to dismiss suffix optimization as “open-model only” ([3](#), [4](#)).

However, the literature also makes clear that transfer is not uniform. Several factors mediate it:

1. **Tokenizer mismatch:** discrete suffixes optimized on one tokenizer may fragment differently on another model.
2. **Chat-template mismatch:** where system, user, and assistant turns are separated can alter the causal pathway by which suffix tokens influence generation.
3. **Safety-head and refusal-style differences:** some models refuse early and rigidly, others only after decoding harmful continuations.
4. **Prompt-position sensitivity:** as newer work shows, token placement itself changes effectiveness, so transfer may degrade if APIs wrap prompts differently ([1](#), [2](#), [3](#)).

A practically important implication is that the most dangerous transferable attacks may not be the most human-readable ones. An unreadable suffix can still be operationally useful if it survives platform filtering and transfers across enough targets. Conversely, readable jailbreaks may be easier to moderate or detect via semantic classifiers. Suffix attacks thus occupy a distinctive corner of the trade space: low readability and high development complexity, but potentially strong white-box efficacy and—under newer universal methods—meaningful black-box portability ([3](#), [4](#), [5](#)).

Comparative evidence on open versus closed model performance

The supplied record supports a consistent asymmetry between open and closed settings. Open-source aligned models are the easiest domain for optimization-based suffix methods because attackers can directly inspect gradients, token embeddings, and output probabilities. Closed models are harder because the attacker must rely on transfer from surrogates or on non-gradient black-box methods. Yet AmpleGCG and related work show that this asymmetry is no longer absolute; open-source optimization can serve as a stepping stone to meaningful attacks on proprietary APIs ([2](#), [3](#), [4](#)).

A useful contrast is provided by the black-box study on explicitly harmful prompts against commercial LLMs. While not a suffix-optimization paper, it demonstrates that one can often obtain jailbreak success against commercial systems without gradients at all, using direct harmful prompting and other black-box strategies ([5](#)). This contextualizes suffix attacks in two ways. First, it means transfer from open-model GCG is not the only path to attacking closed APIs. Second, it implies that suffix optimization competes not with “no attack,” but with other black-box families that may be cheaper and more readable. Therefore, the operational value of a suffix method against commercial models depends on whether it delivers materially higher ASR, stealth, or universality than simpler black-box alternatives.

The literature suggests three circumstances where suffix attacks remain especially valuable despite this competition:

- When a model resists obvious harmful prompts but still contains token-level vulnerabilities exploitable by optimized strings.
- When attackers can precompute or learn reusable suffixes, lowering marginal deployment cost.
- When transfer from open-source surrogates materially outperforms human-authored jailbreaks on particular safety stacks ([3](#), [4](#), [5](#)).

Why these attacks work: alignment failure at the token-distribution level

The central explanatory question is why semantically nonsensical suffixes can override explicit safety training. The literature points toward a mechanistic answer: alignment and refusal behavior are not separable “hard rules,” but conditional behaviors encoded in the same next-token prediction machinery as all other language competencies. If optimization can identify a token pattern that shifts the model’s hidden-state trajectory away from a refusal basin and toward a compliance basin, then the model may produce harmful content even though the prompt’s semantic intent remains harmful throughout ([1](#), [2](#), [4](#)).

This means the attack does not need to “convince” the model in a human sense. It only needs to alter the local probability landscape so that non-refusal continuations become more likely than refusal templates. Several features of aligned chat models make this possible:

1. **Refusal is pattern-based and probabilistic.** Safety tuning teaches common refusal continuations but does not install a symbolic filter over all future token sequences.
2. **Control is distributed, not localized.** Safety behavior emerges from many interacting parameters; optimized suffixes can perturb these interactions indirectly.
3. **Instruction-following competes with refusal.** If optimization boosts features associated with “complete the request” behavior or target-answer mimicry, that pressure may dominate safety preferences.
4. **Template sensitivity creates narrow decision boundaries.** Small token changes can move the model across those boundaries, especially near ambiguous refusal/compliance regions ([1](#), [2](#)).

The position paper strengthens this account by showing that token placement matters. If the same or similar tokens have different effects depending on where they appear, then the mechanism is not simply “a magic suffix token sequence,” but a prompt-wide causal interaction in the decoder’s processing of role structure, recency, and contextual salience. This helps explain why suffixes

sometimes fail under alternate chat wrappers or API formatting, and why they can reappear as effective when inserted in more advantageous positions ([1](#)).

Refusal bypass as objective hacking rather than semantic persuasion

A useful way to interpret GCG-family attacks is as objective hacking against the model’s generation policy. Instead of seeking semantic concealment—as persona attacks often do—or iterative trust-building—as multi-turn attacks do—suffix optimization directly manipulates the probability assigned to desired continuations. The attacker’s optimization target is usually some form of “make the model emit this harmful answer or at least stop refusing.” This is a much more direct attack on the policy boundary than manually crafted social-engineering prompts ([2](#), [4](#)).

This distinction matters for defensive design. A defense that successfully detects obvious role-play framing may still fail on a token sequence with no interpretable malicious semantics. Similarly, refusal fine-tuning may improve average behavior while leaving adversarially reachable pockets in token space. In effect, GCG-family methods reveal that safety tuning can be robust at the level of ordinary natural language variation yet fragile at the level of worst-case discrete optimization ([2](#), [4](#)).

Position, syntax, and attack structure: beyond “append random garbage”

The paper “Beyond Suffixes: Token Position in GCG Adversarial Attacks on Large Language Models” is especially valuable because it corrects an oversimplification common in practitioner discussions. The phrase “adversarial suffix” suggests that success arises because a fixed string is appended at the end of a prompt. The paper instead shows that position is itself a salient variable and that attack efficacy can depend on where optimized tokens intervene in the prompt sequence ([1](#)).

This has four implications. First, benchmark results based only on terminal suffixes may underestimate the broader reachable attack surface. Second, defenses that strip or inspect trailing gibberish may miss structurally similar attacks inserted elsewhere. Third, transferability analyses must account for prompt formatting and role wrappers, since these alter token positions relative to system prompts and user requests. Fourth, the attack family begins to resemble a more general prompt-structure optimization problem rather than a single suffix-only trick ([1](#)).

For the larger report’s focus on defense bypass, this position sensitivity helps explain why straightforward safeguards fail. Many guardrails are built around semantic inspection of user intent or detection of suspicious suffix-like artifacts at predictable locations. If attack efficacy depends on deeper sequence interactions, then superficial pattern filters will be incomplete by design. A model may internally process the same harmful instruction differently depending on token placement, role

boundaries, and recency effects, allowing optimized positions to route around defenses that were tuned to typical prompt layouts ([1](#), [2](#)).

Index gradients and richer optimization signals

The index-gradient paper marks a methodological maturation of the field. Classic GCG already uses gradient information, but index-gradient methods seek a more informative signal about *which* token substitutions will most effectively change the model’s output distribution. The paper explicitly frames this as exploiting “index gradients” for optimization-based jailbreaking, and situates it alongside GCG, Probe Sampling, MAC, and AmpleGCG ([2](#)).

The deeper significance is that first-generation GCG may have been limited not because optimization-based jailbreaking was inherently weak, but because it used an imperfect local search heuristic over a difficult discrete space. If richer gradient proxies better capture the influence of token replacements on target outputs, then the family can improve on both speed and efficacy without changing the fundamental threat model. This is analogous to exploit development in other domains: a technique once dismissed as too expensive or brittle can become practical once the search procedure improves.

In the supplied paper’s framing, the benefits are at least threefold:

- **More efficient coordinate selection:** fewer wasted candidate evaluations.
- **Potentially better escape from poor local optima:** improved success at fixed budget.
- **Compatibility with other attack layers:** index-based optimization can complement acceleration and generative methods rather than replace them ([2](#)).

This interoperability matters. The future of suffix attacks is unlikely to be a single dominant algorithm. More plausibly, attackers will use hybrid pipelines: index-gradient methods to generate strong seeds, probe sampling to reduce cost, multi-surrogate optimization for transfer, and a learned suffix generator to scale deployment. The literature already points in this direction by discussing these methods in a shared design space rather than as isolated papers ([2](#), [4](#)).

Measured effectiveness, cost, and access requirements across the main suffix subfamilies

The table below synthesizes the empirical and operational picture supported by the supplied sources. It is intentionally conservative where exact percentages vary across papers.

Subfamily	Source-supported effectiveness claim	Development cost	Required expertise	Deployment cost	Access requirement	Transfer outlook
Vanilla GCG	Very strong white-box baseline on aligned LLMs (2 , 4)	High	High	Low once suffix is found	Full gradients on source model	Historically limited to moderate
Faster/accelerated GCG-style search	Preserves optimization-based strength while reducing search burden (2 , 4)	Medium–High	High	Low	Full gradients on source model	Similar or somewhat improved via broader search budget
Position-aware optimization	Shows untapped gains from placement changes beyond end-suffix baselines (1)	Medium–High	High	Low	Full or close white-box evaluation	Transfer sensitive to prompt wrappers
I-GCG / index-gradient methods	Improves optimization quality or efficiency over prior gradient baselines in reported experiments (2)	Medium–High	High	Low	White-box gradients / probabilities	Better source performance; transfer still not guaranteed
	Optimization accelerators or	Medium	High	Low	White-box or privileged	

Subfamily	Source-supported effectiveness claim	Development cost	Required expertise	Deployment cost	Access requirement	Transfer outlook
Probe Sampling / MAC	quality enhancers discussed in same methodological lineage (2)				source access	Mostly source-side benefits
AmpleGCG / learned universal generators	Produces universal and transferable suffixes for open and closed LLMs (3)	High upfront	Very high to develop, lower to operate	Very low	White-box surrogate corpus for training; black-box at use time	Strongest transfer-oriented evidence in supplied set

What stands out is the split between **research sophistication** and **operator sophistication**.

Developing AmpleGCG-like systems or improved gradient objectives is an expert activity. Using the resulting suffixes or generators may be far easier. This asymmetry is critical for real-world risk: once a high-skill group produces a universal attack artifact, low-skill users can potentially reuse it at scale ([3

Tool-, Agent-, and Ecosystem-Level Jailbreak Dynamics

Measurement boundary, evidence classes, and how this section differs from earlier coverage

While earlier sections already addressed **agentic orchestration of the jailbreak conversation itself** and the broad existence of **tool/agent-mediated exploit chains**, this section is narrower in one sense and broader in another. It is narrower because it does **not** re-cover Crescendo-style conversational escalation, Tree-of-Attacks search, or MRJ-Agent as prompt-strategy mechanisms. It is broader because it evaluates the **full socio-technical pipeline** in which jailbreaks become operational capability: prompt repositories, public sharing and remix, browser-enabled agents, retrieval-augmented generation (RAG) systems, policy-routing layers, and autonomous attack

improvement loops. The emphasis is therefore on **system exploitation dynamics and attacker economics**, rather than on the prompt family mechanics already treated elsewhere. This distinction is important because a prompt that is only moderately effective in isolation can become operationally dangerous when embedded in a tool-using agent, a long-context workflow, or a public market that amortizes search cost over thousands of users ([1](#), [2](#), [3](#)).

The supplied record is uneven across this topic. Some components are directly documented in the cited literature, especially the transition from single-turn to multi-turn and agent-driven jailbreaks, the growing relevance of prompt injection and retrieval-mediated compromise, the existence of in-the-wild jailbreak corpora, and the increasing automation of adversarial search ([4](#), [5](#), [6](#), [7](#)). By contrast, quantitative estimates for “jailbreak-as-a-service” pricing, underground market size, or browser-agent exploit-chain success rates are sparse in the supplied sources and remain less standardized in the open literature than text-only attack success rate (ASR) benchmarks. Accordingly, this section explicitly separates:

1. **Documented evidence**: statements directly supported by the supplied sources.
2. **Cross-source synthesis**: claims that integrate multiple supplied sources into a system-level interpretation.
3. **Inference under uncertainty**: judgments about likely attacker economics or ecosystem evolution where the open record remains incomplete as of April 2026.

This matters methodologically. ASR figures from HarmBench-like evaluation settings and papers on multi-turn jailbreaking can be compared, albeit imperfectly, because they target refusal bypass under curated task sets. Tool- and ecosystem-level attacks, however, often target **capability execution** rather than only harmful text emission. A browser agent may be “jailbroken” if it autonomously navigates to sensitive resources, persists unsafe context, or transforms retrieved content into prohibited assistance, even when the base model still occasionally emits refusals. Similarly, a RAG pipeline can fail without the base model being conventionally “jailbroken” in the one-prompt sense if retrieval, ranking, chunking, and instruction hierarchy together induce unsafe behavior. Therefore, the central dependent variable here is not merely refusal bypass but **end-to-end exploit realization**.

The supplied sources support several key premises for this section:

- Public prompt-sharing ecosystems for jailbreaks are plainly visible, including curated GitHub topic pages and repositories dedicated to collections of jailbreak prompts, which confirms a durable open circulation layer for attack artifacts ([1](#), [2](#), [3](#)).

- Formal benchmarking work had already expanded by 2024 to prompt injection attacks and defenses, signaling that system-level exploitability beyond plain direct prompting had become a first-class evaluation problem (7).
- Multi-turn and agent-driven decomposition attacks were explicitly documented by 2024–2026 papers, including references to “agent-driven multi-turn decomposition jailbreaks,” “MRJ-Agent,” and “Echo Chamber,” showing that prompt development was moving toward software-mediated automation rather than remaining a purely manual craft (5).
- Automated adversarial search was also accelerating in suffix-optimization work, including GCG variants, Faster-GCG, and generative or transfer-oriented methods, which is economically relevant because it lowers the cost of discovering reusable jailbreak payloads (7).

A practical implication follows: the most consequential shift from 2023 to 2026 may not be that any one jailbreak family became universally dominant, but that **attack-development labor became decomposable**. Discovery, validation, packaging, transfer testing, public dissemination, and deployment can now be separated and scaled by different actors. That decomposition is the core of jailbreak ecosystem maturation.

Browser agents, toolchains, and RAG as multiplicative exploit surfaces

The most important system-level change in the 2024–2026 period is that jailbreaks increasingly matter because they are attached to **action loops** rather than merely to text generation. A base model that outputs disallowed text is harmful; a model that can browse, retrieve, summarize, operate external tools, or compose those actions into task execution is qualitatively more dangerous. This is not simply because it can do “more things,” but because tools alter the **attack surface geometry**. They introduce external state, hidden context, retrieved instructions, latent command channels, and opportunities for decomposition that are absent in a single prompt-response exchange.

The supplied sources do not provide a single benchmark paper devoted exclusively to browser-agent exploit chains, but they contain enough evidence to support a structured account. The taxonomy source already recognized tool/agent-mediated exploit chains as a distinct family, while acknowledging the relative scarcity of uniformly benchmarked evidence compared with prompt-only attacks. The prompt-injection benchmarking citation in [7] is highly relevant here because browser agents and RAG systems are the natural operational setting in which prompt injection becomes exploit execution rather than abstract misalignment. In other words, prompt injection is the mechanism; the browser or retriever is the delivery and persistence substrate.

Browser-agent jailbreak dynamics

A browser-enabled agent differs from a chat endpoint in at least five security-relevant ways:

1. **Indirect prompt acquisition:** it ingests instructions from webpages, documents, or rendered tool outputs, not only from a user’s visible message.
2. **Instruction hierarchy ambiguity:** the system must decide whether retrieved or page-embedded instructions are subordinate content or actionable directives.
3. **State persistence:** prior page content, browsing notes, hidden scratchpads, or planner memory may continue influencing later steps.
4. **Action externalization:** harmful model outputs can be converted into navigation, search, retrieval, code execution, or transaction proposals.
5. **Delegation opacity:** unsafe intermediate reasoning may be concealed inside agent planning even if the final natural-language output is partially sanitized.

The significance of these properties for jailbreak dynamics is best understood as a **multiplicative** rather than additive effect. A mediocre prompt-only jailbreak can become effective if the browser retrieves supporting context that normalizes, reframes, or “legitimizes” the request. Conversely, even when a direct prompt is refused, a model may become vulnerable after exposure to staged external content that gradually modifies context over several tool invocations. This is strongly analogous to multi-turn escalation, but the “turns” are now partly supplied by external artifacts and environment responses rather than by a human interlocutor. The Echo Chamber framing is particularly useful conceptually: it foregrounds how cumulative context reinforcement can normalize unsafe framing over time. In a browser setting, the chamber can be built not only by the attacker’s dialogue turns but by pages, snippets, summaries, and tool-returned observations that all point in the same unsafe direction (5).

The literature on multi-turn context jailbreaks and active fabrication is relevant because browser-agent compromise often relies on **decomposition plus contextual reinforcement**, not on a single overtly malicious request. A browsing agent can be induced to collect harmless-seeming fragments, each individually permissible, and then synthesize them into a prohibited end state. This resembles compositional prompt attacks but differs operationally in that the model is outsourcing parts of the attack to the external information environment. The [OpenReview paper on global refinement and active fabrication](#) explicitly situates recent work within the shift from single-turn explicit prompts toward more concealed, multi-turn forms, and that logic extends naturally to browser-mediated workflows where harmful content can be distributed across steps and sources (6).

Defensively, browser-agent attacks bypass several mechanisms that are stronger in plain chat settings:

Defensive layer	Why it is more effective in plain chat	Why browser-agent workflows weaken it
Surface prompt classifiers	User text is the primary visible input	Malicious instructions may arrive through retrieved pages or tool output rather than the user message
Static jailbreak signature matching	Reusable strings or known templates can be blocked	Browser content can generate fresh, context-specific phrasing on each run
Refusal training on direct harmful asks	Unsafe intent is often explicit	Intent can be decomposed into apparently benign browsing subtasks
Rate limiting on chat turns	Fewer direct interactions constrain search	Tools create additional “hidden” interaction opportunities within one high-level user request
Human review of prompts	User-facing content is inspectable	Planner state, hidden notes, and retrieved content can be difficult to audit in real time

No supplied source gives a standardized cross-vendor ASR table for browser-agent exploit chains analogous to GCG benchmarks. That absence is itself a notable empirical finding: by April 2026, the field remains more mature in measuring **text-level jailbreaks** than **agent-level exploit completion**. What can be said with confidence is that the literature’s center of gravity had already moved toward multi-step, agent-mediated, and decomposition-heavy attacks, and this shift is exactly what browser agents operationalize ([5](#), [6](#)).

RAG-mediated exploit chains

RAG systems deserve separate treatment because their failure modes differ from browser agents even though both ingest external text. In a browser-agent setting, the model is often asked to act in an environment. In RAG, the model is asked to answer using retrieved evidence. The security problem is that retrieval pipelines convert **documents into privileged context**. Once malicious or adversarially chosen text is admitted into the context window, the model may overweight it, especially when the system prompt encourages deference to retrieved materials.

The supplied source set includes a direct pointer to formalized prompt injection benchmarking and defenses ([7](#)). Although that citation is not itself a full RAG-specific field survey, it is highly salient because prompt injection in deployed systems most often manifests through retrieved content,

attached files, embedded instructions in knowledge bases, or tool output. RAG-mediated jailbreak chains typically involve one or more of the following patterns:

- **Instruction smuggling in retrieved passages:** a document chunk contains hidden or overt instructions that the model treats as authoritative.
- **Context poisoning through ranking/manipulation:** the retriever is steered toward adversarial content whose main function is not factual relevance but behavioral steering.
- **Distributed intent assembly:** no single retrieved passage is obviously unsafe, but the aggregate context supports a prohibited synthesis.
- **Chain-of-summaries attenuation failure:** downstream summarization strips explicit warning cues while preserving harmful procedural substance.
- **Policy confusion:** the model interprets the retrieved source as part of a higher-priority instruction layer rather than as quoted evidence to be analyzed skeptically.

From a jailbreak-evasion standpoint, RAG can bypass prompt-level safeguards because the harmful steering is not concentrated in the user prompt. It can also bypass some output-side moderation if the system frames the response as “summarizing retrieved content” or “following source materials,” thereby reducing the salience of intent signals that ordinary refusal models rely upon. Moreover, RAG complicates responsibility assignment inside the stack: is the exploit due to the retriever, the chunker, the ranking objective, the system prompt, the model, or the post-processing layer? Attackers benefit from this ambiguity because each control may appear individually reasonable while the overall pipeline is unsafe.

This problem intersects directly with long-context research. As context windows expanded, the opportunity for retrieval-mediated steering increased, not only because more adversarial material could fit in the prompt but because defenders often assumed that longer context would improve groundedness. In practice, longer context can also amplify **context dilution and instruction confusion**, making it easier to bury harmful constraints or instructions inside large retrieved corpora. The citation to Babilong in the supplied material is not a jailbreak paper per se, but it is still informative: long-context reasoning under distraction is difficult, and that limitation bears on RAG security because models must identify which tokens are relevant, which are instructions, and which are adversarial noise ([8](#)).

A chain-level view: from prompt to exploit

The most security-relevant unit of analysis is therefore the **exploit chain**, not the isolated prompt. The following table synthesizes how chain elements interact:

Chain stage	Typical attacker input	System component abused	Bypassed safeguard class	Operational consequence
Initial request framing	Benign-seeming task or role	User interface / task planner	Intent classifiers tuned for explicit harmful asks	Low-friction entry point
Retrieval or browsing step	Search query, URL, or knowledge fetch	Browser, retriever, ranking layer	Surface moderation focused on user prompt only	External adversarial content enters trusted context
Context reinforcement	Multi-step notes, summaries, or quoted passages	Memory, scratchpad, long-context assembly	Per-turn moderation and static prompt scanning	Unsafe framing becomes normalized over time
Instruction reinterpretation	“Follow source,” “continue workflow,” “complete task”	System prompt hierarchy resolution	Direct refusal training on user text	Retrieved content exerts hidden control
Harmful synthesis or action	Summarization, transformation, navigation, or tool call	Base model plus tool executor	Output moderation focused on final text only	Textual refusal may be bypassed or replaced by action execution

This chain framing helps explain why the economics of system-level attacks differ from ordinary promptcraft. The attacker no longer needs a universal direct jailbreak if they can instead engineer conditions under which the target stack **self-supplies the persuasive context**. That makes browser and RAG systems especially concerning: they transform the model’s environment into part of the attack surface.

Public sharing, community markets, and the maturation of jailbreak supply chains

This subsection is intentionally different from the earlier discussion of in-the-wild prevalence of manual jailbreaks. The earlier material established that prompt sharing was widespread and that public communities reduced attacker entry cost. Here the focus is the **supply-chain structure** that emerges when public repositories, social discovery, benchmark publication, and automated search methods interact. The question is not only whether jailbreak prompts are shared, but how sharing changes the **organization of adversarial innovation**.

The supplied sources provide direct evidence that jailbreak artifacts are openly indexed and discoverable on mainstream infrastructure. GitHub topic pages for **jailbreak-prompts**, **ai-jailbreak**, and related labels show that repositories are organized, tagged, and exposed through ordinary developer search mechanisms rather than only through hidden or illicit channels ([1](#), [2](#), [3](#)). This matters because mainstream discoverability lowers acquisition cost, shortens learning time, and promotes template recombination. It also blurs the line between research artifact, hobbyist experimentation, and offensive resource circulation.

From isolated prompts to reusable attack products

A mature attack ecosystem usually exhibits five features:

1. **Artifact standardization**: prompts, suffixes, templates, and scripts are packaged for reuse.
2. **Search cost amortization**: one actor discovers an effective artifact; many others redeploy it.
3. **Versioning and mutation**: attacks are updated after defenses or model refreshes.
4. **Specialization**: different participants focus on discovery, packaging, distribution, or operational use.
5. **Feedback loops**: deployment outcomes inform the next generation of artifacts.

The public jailbreak repositories and topic pages are evidence of at least the first three features. The optimization and benchmark papers suggest the latter two are increasingly plausible. For example, Faster-GCG, AmpleGCG, and transfer-learning work on adversarial suffixes indicate that discovery is becoming more systematic and that some attack payloads can be learned, generalized, and transported across targets ([7](#)). Once suffixes or prompt scaffolds become reusable artifacts rather than one-off discoveries, the ecosystem starts to resemble a **lightweight exploit market**.

It is important to avoid overclaiming here. The supplied sources do **not** provide direct financial market data such as recurring subscription prices, seller counts, or revenue estimates for a formal underground “jailbreak-as-a-service” sector. However, they do support a more modest but still

significant conclusion: by 2024–2026, the prerequisites for such a market were present in public form. These prerequisites include open distribution channels, benchmarked performance claims, automation methods for artifact generation, and a user base that can copy and redeploy prompts at near-zero marginal cost.

That public market logic can be described in economic terms:

- **Discovery cost** is paid by researchers, hobbyists, or specialized attackers who design or search for jailbreaks.
- **Validation cost** is paid when artifacts are tested across models or benchmark sets.
- **Packaging cost** is paid when artifacts are converted into shareable prompts, scripts, APIs, or repositories.
- **Distribution cost** is minimal on mainstream platforms.
- **Deployment cost** is mostly token/API spend and may be negligible for local or open models.
- **Maintenance cost** arises when providers patch defenses or new model versions reduce efficacy.

This decomposition matters because it reveals why public sharing is so powerful. The expensive step is discovery; once paid, the artifact can diffuse rapidly. The same logic already appeared in earlier sections for manual prompts, but at ecosystem scale the effect is stronger because automation makes the discovery step itself more reproducible.

Jailbreak-as-a-service as a service model, not merely a marketplace label

“Jailbreak-as-a-service” should not be understood narrowly as a dark-web storefront. In the 2023–2026 LLM context, the more analytically useful definition is **any service arrangement that externalizes jailbreak discovery or deployment labor for downstream users**. Under that definition, several layers qualify:

Service layer	Typical offering	Technical sophistication	Marginal cost to user	Evidence level in supplied sources
Static prompt collections	Reusable jailbreak prompt lists	Low	Near-zero	Directly supported by GitHub topic pages and public repositories (1)
Parameterized prompt templates	Fill-in-the-blank attack scaffolds	Low to moderate	Near-zero to low	Inferred from public prompt-sharing structure

Service layer	Typical offering	Technical sophistication	Marginal cost to user	Evidence level in supplied sources
Automated search scripts	Code that generates or optimizes prompts	Moderate	Low once obtained	Supported by citations to automated attack literature (4 , 7)
Agentic jailbreak planners	Systems that conduct multi-round attacks	Moderate to high	Lower human skill per deployment	Supported by agent-driven multi-turn attack references (5)
Managed offensive APIs or wrappers	Third party handles discovery/rotation/deployment	High	Usage fee likely, though not quantified in supplied sources	Plausible inference, not directly measured

The service-model perspective reveals a critical security asymmetry: defenders must secure the entire stack continuously, whereas attackers can consume a service that hides complexity. This lowers the effective skill threshold for end users even if the upstream developers are sophisticated. Public repositories already perform some of this function informally. A curated collection of working prompts is a rudimentary service because it saves users the cost of original discovery. Search scripts and optimization code provide a higher tier. Agentic wrappers provide a still higher tier by converting adversarial strategy into an executable workflow.

Community incentives and the role of benchmark culture

Another underappreciated driver of the jailbreak ecosystem is that benchmark publication and public leaderboard thinking can unintentionally function as **market signaling**. HarmBench and related evaluation efforts are valuable for defenders, but they also standardize the language of efficacy and create a public vocabulary for comparing attack quality ([4](#)). Similarly, optimization papers that advertise stronger transfer, faster search, or better universality naturally inform adversaries which techniques offer the highest return on effort ([7](#)).

This does not imply that benchmarking research causes misuse; rather, it means the ecosystem now has some features familiar from dual-use security research:

- published methods,
- measurable outcomes,
- reusable code or pseudocode,
- cross-target comparison,
- and incentives to improve efficiency.

Once those ingredients exist, community markets become more credible because buyers or users can compare artifacts by reported performance, stealth, transferability, or query budget. This is especially true for automated prompt development methods. A suffix-optimization artifact or an agentic planner can be marketed—formally or informally—not just by anecdote but by benchmarked efficacy.

Real-world observability constraints

Although the ecosystem clearly exists in public form, its darker or commercialized layers remain difficult to measure rigorously. Three reasons stand out:

1. **Reputational migration:** as moderation and platform governance increase, communities may shift from public repos to invite-only channels.
2. **Ephemeral artifacts:** prompts and wrappers can be deleted, renamed, or quickly rehosted.
3. **Attribution ambiguity:** a repository may frame itself as “research,” “prompt engineering,” or “uncensored AI,” making offensive intent difficult to classify consistently.

The GitHub topic pages nonetheless reveal a semantically suggestive environment, including adjacency to tags such as “uncensored-ai,” “wormgpt,” “blackhat-gpt,” and “dark-llm” ([1](#), [3](#)). These labels should not be overread as proof of criminal use, but they are relevant ecosystem indicators. They show that jailbreak prompt sharing is embedded in a wider discursive market around “uncensored” and offensive-use LLM branding. For defenders, this means monitoring only formal academic papers will miss a substantial portion of adversarial innovation pathways.

The economics of automated adversarial prompt development

This subsection deliberately avoids re-covering the attack-family mechanics of GCG and related suffix optimization, which were addressed previously. Instead, it examines the **economic consequences** of those methods for the broader jailbreak ecosystem: how automation changes cost curves, labor allocation, and the feasibility of continuous attack adaptation.

The main economic question is straightforward: **when does automated adversarial prompt development become cheaper than manual promptcraft, and for whom?** The answer differs depending on whether one considers initial development, cross-model transfer, deployment at scale, or long-term maintenance.

Cost decomposition across the adversarial development pipeline

A structured way to compare manual and automated development is to separate fixed and variable costs.

Cost component	Manual/public prompt reuse	Automated search/ optimization	Agentic autonomous evolution
Initial expertise required	Low to moderate for reuse; higher for original invention	Moderate to high	High for builder, low for downstream operator
Compute requirement	Minimal	Potentially substantial during search/training	Moderate to high depending on loop design
Query/API budget	Low for simple prompts	Often high during optimization or evaluation	Potentially high but amortizable
Transfer testing burden	Mostly empirical trial-and-error	Can be built into optimization/ benchmark loop	Can be automated continuously
Marginal deployment cost	Very low	Low once artifact found	Low once system deployed
Adaptation to new models/patches	Manual editing or replacement	Re-run optimization or fine-tune generator	Continuous online adaptation possible
Skill barrier for end user	Very low if artifact is shared	Low if tool is packaged	Very low if exposed as a service

The supplied sources support each column to varying degrees. Public prompt-sharing ecosystems clearly lower end-user deployment cost ([1](#), [2](#)). Automated search appears in Tree of Attacks and the

adversarial suffix literature, including Faster-GCG and transfer-oriented methods (4, 7). Agent-driven multi-turn decomposition and related references support the plausibility of still more automated attack loops (5).

A central economic insight is that automation does **not** need to be cheaper than manual prompting in absolute terms to be strategically important. It only needs to be cheaper at the margin for the attacker's objective. For example:

- If a public prompt collection already works “well enough,” automated optimization may be unnecessary for casual misuse.
- If a defender patches known templates frequently, automation becomes attractive because it can discover replacements faster than humans can iterate manually.
- If the attacker targets many models or many harmful behaviors, automation becomes more economical because transfer and batch evaluation amortize the search cost.
- If the attacker sells access or provides a managed service, the fixed development cost can be spread across many customers.

This is why automated adversarial prompt development has outsize strategic significance even before it dominates common deployment. Its value is highest when defenders become more active.

Public sharing as subsidy for autonomous search

An especially important and under-discussed point is that public prompt repositories and benchmark corpora function as a **training subsidy** for automated adversarial systems. Shared prompts are not merely deployment artifacts; they are also data. A model or search algorithm can use public jailbreak collections as seeds, templates, priors, or evaluation scaffolds. Thus, the transition from manual public sharing to autonomous attack evolution is not discontinuous. It is a pipeline:

1. Humans invent prompts.
2. Communities collect and label them.
3. Researchers benchmark them.
4. Optimization methods generalize them.
5. Agents deploy and refine them adaptively.

The GitHub prompt-sharing pages establish the visible existence of stage 2; the benchmark and optimization papers show stages 3 and 4; the multi-turn agent papers suggest stage 5 is emerging (1, 4, 5, 7). Economically, this pipeline is powerful because each stage reduces uncertainty for the next. An optimizer does not begin from random strings; it begins from a socially curated attack manifold. An agentic planner does not need to invent all strategy from scratch; it can iterate on known decomposition motifs.

This dynamic parallels exploit development in other domains, where proof-of-concept publication reduces the cost of weaponization. The difference in LLM security is that the artifacts are often **language-native**, easier to mutate, and cheaper to distribute. A prompt collection can be copied instantly, paraphrased automatically, translated across languages, or embedded into wrappers with negligible cost.

Search efficiency and transfer as economic multipliers

The acceleration trend in suffix optimization deserves to be read economically, not only technically. Faster-GCG lowers the resource cost of discrete optimization; generative and transfer-oriented methods such as AmpleGCG aim to learn broader distributions of effective suffixes rather than optimizing one artifact at a time (7). Each of these improvements alters attacker economics in a specific way:

- **Faster search** lowers the cost of discovery per artifact.
- **Better transfer** lowers the number of separate discovery runs needed across models.
- **Universal or reusable artifacts** lower the cost per harmful behavior.
- **Learned generators** convert search from per-instance optimization to reusable model inference.
- **Checkpoint-aware or defense-aware methods** reduce the need to rediscover attacks after each defensive update.

These are multiplicative gains. If search becomes 5–10× faster, transfer cuts the number of target-specific runs in half, and public sharing eliminates downstream development for most users, the total attack-development burden falls by much more than any one factor alone would suggest. The field’s technical focus on ASR sometimes obscures this: two attacks with similar success rates may differ radically in strategic threat if one is expensive, brittle, and bespoke while the other is fast, reusable, and easily packaged into a service.

Labor substitution: from prompt engineers to attack operators

A mature ecosystem also changes who performs the work. In the earliest public jailbreak waves, a large share of offensive capability depended on individuals willing to experiment manually with personas, roleplays, and reframing. By 2024–2026, the literature suggests a transition toward **labor substitution**:

- prompt engineers are replaced by optimization scripts,
- conversational tacticians are replaced by agentic planners,
- and ad hoc sharing is replaced by repositories and reproducible pipelines.

This substitution lowers the average skill requirement for deployment while increasing the peak sophistication available to organized actors. It also creates a stratified market:

Actor type	Likely capabilities	Primary cost	Likely source of artifacts
Casual end user	Reuse static prompts, simple wrappers	Token spend	Public repositories
Enthusiast operator	Run open-source search scripts, tweak templates	Moderate compute and time	Academic code, GitHub repos
Service provider	Maintain prompt rotation, transfer testing, wrappers	Engineering labor, infrastructure	Combination of public research and proprietary iteration
Advanced adversary	Continuous autonomous adaptation, tool-mediated chains	Higher R&D and integration cost	Internal pipelines plus public seed artifacts

This stratification helps explain how jailbreak ecosystems can expand without every participant becoming technically advanced. As in many cyber domains, specialization and packaging lower the operational barrier.

Autonomous jailbreak evolution, model-to-model adaptation, and defense pressure

This final subsection differs from earlier discussions of transferability and agentic orchestration by focusing on **continuous adaptation under defensive pressure**. The key issue is not whether a single attack transfers across models at one point in time, but whether the ecosystem can **evolve attacks faster than providers can harden defenses**.

From static prompts to adaptive attack loops

Static prompt sharing represents the simplest evolutionary regime: users circulate artifacts until they stop working. Automated optimization introduces a second regime: artifacts can be regenerated after failure. Agentic multi-turn systems suggest a third regime: the attacker can deploy a planner that **observes refusals, diagnoses failure modes, and revises strategy online**. This is the beginning of autonomous jailbreak evolution.

The supplied sources do not present a single end-to-end commercial system that fully automates this cycle, but they provide the necessary ingredients:

- iterative multi-turn refinement and context shaping (5, 6);
- automated black-box search over attack paths (4);
- fast and transferable suffix generation (7);
- and public prompt corpora usable as seeds or priors (1).

When combined, these ingredients support an adaptive loop of the following form:

1. Seed the system with known jailbreak templates or suffixes.
2. Probe a target model or agent stack.
3. Observe refusals, truncations, or moderation triggers.
4. Modify phrasing, decomposition strategy, or attack payload.
5. Retest across behaviors or models.
6. Preserve successful variants for future use or sharing.

This loop is economically significant because it transforms attack development from a one-time craft exercise into a **continuous optimization process**. If integrated with public repositories and evaluation harnesses, the loop can accumulate improvements over time. In practice, even relatively simple automation can outperform humans in persistence, coverage, and consistency.

Defensive updates and attacker adaptation horizons

A critical but often under-specified variable is the **adaptation horizon**: how long does it take an attacker to recover efficacy after a provider update? Precise cross-vendor measurements are rare in the supplied record, but the structure of the literature permits several grounded observations.

First, attacks that rely on a single recognizable string or known public template may degrade quickly after targeted patching. Second, attacks that derive power from more general model tendencies—context accumulation, instruction confusion, decomposition, retrieval trust, or optimization over token distributions—are harder to extinguish because they exploit broad behavioral properties rather than one exact phrase. This is why system-level and autonomous attacks are strategically more concerning than any one meme prompt. They are less dependent on a fixed artifact and more dependent on the persistent geometry of the model-stack interaction.

Third, transfer-oriented optimization methods imply that attackers do not need perfect model-specific recovery. If they can maintain a portfolio of partially effective attacks across families, they can continue operating while refining newer variants. Public sharing further shortens recovery time

because patch impact can be crowdsourced: users quickly discover which prompts still work and which require mutation.

The net result is that defensive patching may impose **maintenance costs** on attackers without necessarily restoring a stable advantage to defenders. This dynamic is familiar from adversarial ML generally, but LLM ecosystems intensify it because:

- artifacts are cheap to mutate,
- evaluation is cheap to rerun,
- transfer can be nontrivial,
- and public communities accelerate information diffusion.

Cross-model adaptation and wrapper diversity

Another important evolutionary factor is that deployed systems increasingly involve **wrapper diversity** rather than only base-model diversity. Two products may use similar frontier models but differ in system prompts, tool policies, browsing behavior, retrieval stack, rate limits, and moderation layers. This diversity cuts both ways.

For defenders, wrapper diversity can reduce naïve transfer of a single prompt artifact. For attackers, however, it creates an incentive for **meta-level attack development** aimed not only at model tokens but at common workflow patterns: retrieve-then-summarize, browse-then-act, cite-source-then-answer, or split-task-into-subtasks. Agentic and decomposition-based attacks are especially well suited to such diversity because they can adapt strategy at runtime rather than relying on exact token-level transfer.

This is one reason the ecosystem is likely to continue shifting toward autonomous and semiautonomous methods. Static prompt libraries scale poorly across heterogeneous wrappers; adaptive planners scale better. Likewise, RAG-mediated and browser-mediated exploit chains target general architectural habits—trusting retrieved text, preserving context, following task decompositions—rather than one vendor’s exact alignment string.

Security implications for defenders

The system-level threat model emerging from the 2023–2026 literature is therefore not merely “users may type bad prompts.” It is closer to the following:

- open communities produce and share attack seeds;
- automated methods transform seeds into scalable artifacts;
- tool-using systems supply external context that amplifies weak attacks;

- adaptive agents convert failures into further search steps;
- and heterogeneous wrappers create many exploitable seams even when base models improve.

The defensive implication is that prompt hardening alone is inadequate. Several controls become more important in tool/agent settings:

Defense priority	Why it matters specifically here	Limitation
Strict instruction/data separation in retrieved content	Reduces RAG prompt-injection risk	Hard to guarantee semantically
Tool-call authorization and policy gating	Prevents unsafe action execution despite partial jailbreak success	Can be bypassed if policy model shares vulnerabilities
External-content provenance and trust scoring	Helps downgrade adversarial webpages/documents	Attackers can imitate trusted formatting
Memory/scratchpad minimization or isolation	Limits context reinforcement across steps	May reduce task quality and user utility
End-to-end exploit-chain evaluation	Measures actual system compromise rather than prompt-only ASR	Expensive, environment-specific, less standardized
Continuous red teaming with adaptive attackers	Matches attacker evolution tempo	Resource intensive

The formal benchmarking of prompt injection attacks and defenses in [7] is especially important because it pushes evaluation beyond the model’s response to a naked user prompt and toward the system conditions that make real exploit chains possible (7). Yet a major measurement gap remains: unlike HarmBench-style prompt evaluations, the field still lacks a universally adopted suite for browser-agent and RAG exploit completion that captures retrieval poisoning, latent instruction override, tool misuse, and multi-step harmful synthesis in a single comparable framework.

Empirical status table: what is measured, what is inferred, and what remains uncertain

To make the evidence boundary explicit, Table VI summarizes the state of knowledge from the supplied record.

Claim area	What can be stated confidently	Quantitative support in supplied sources	Confidence
Public jailbreak prompt sharing exists at visible scale	GitHub indexes dedicated jailbreak-prompt repositories and related categories	Qualitative platform evidence; no revenue figures	High (1 , 2 , [3] (https://github

Conclusion

From 2023 to 2026, the jailbreak landscape evolved from low-cost manual persona prompts into a diversified attack ecosystem spanning multi-turn escalation, many-shot context exploitation, optimization-based suffixes, multimodal triggers, and agent/tool-mediated exploit chains. The strongest overall pattern is that no single family dominates across all settings: persona and compositional prompts remain cheap, readable, and widely reused; multi-turn attacks such as Crescendo-style escalation achieve strong black-box effectiveness against frontier chat systems; GCG-derived methods remain among the most powerful in white-box settings but historically face transfer and deployability limits; and newer universal or amortized methods seek to turn expensive research attacks into reusable artifacts. The clearest documented benchmark anchors in the supplied sources are the >95% ASR reported for Compositional Instruction Attack against GPT-4, GPT-3.5, and Llama2-70B, the >70% success rates reported for simplified multi-turn jailbreaks against GPT-4-, Claude-, and Gemini-class models, and the reported 37.74% improvement over the strongest baseline for activation-guided local editing, alongside evidence that universal multimodal jailbreaks and learned suffix generators are improving transferability across targets ([Li et al., 2025](#); [Multi-Turn Jailbreaks Are Simpler Than They Seem, 2025](#); [Wang et al., 2025](#); [AmpleGCG, 2024](#); [UltraBreak, 2026](#)).

The most important implication is that practical risk is increasingly driven not just by raw ASR, but by attacker economics: reusable prompt libraries, public sharing ecosystems, automated search, and agentic orchestration reduce skill requirements and amortize development cost across many users and targets. At the same time, defenses remain fragmented. Surface prompt filters are routinely bypassed by compositional, many-shot, and multi-turn attacks; white-box optimization exposes weaknesses in alignment at the token-distribution level; and tool/RAG/agent deployments expand the attack surface from refusal bypass to end-to-end exploit execution. The report therefore suggests that next-step defenses should prioritize cost-normalized and sequence-aware evaluation, explicit no-attack baselines, transfer and best-of-(N) reporting, and system-level testing for retrieval, tool use, and memory persistence, while moving beyond static prompt blocking toward intent-aware,

telemetry-based, and workflow-level controls. Just as importantly, future research should close the measurement gap for agent/browser exploit chains and adversarial prompt markets, where ecosystem evidence is strong but standardized quantitative benchmarking still lags behind text-only jailbreak evaluation ([SelfGrader, 2026](#); [JailbreakHub, 2024](#); [Benchmarking Prompt Injection Attacks and Defenses, 2025](#); [Echo Chamber, 2026](#)).

References

- <https://arxiv.org/pdf/2405.09113>
- <https://arxiv.org/html/2403.14725v2>
- <https://arxiv.org/html/2404.01833>
- <https://arxiv.org/html/2501.14250v2>
- <https://arxiv.org/html/2507.04365v1>
- <https://arxiv.org/html/2410.15645v1>
- <https://arxiv.labs.arxiv.org/html/2402.10601>
- <https://arxiv.org/html/2409.20089v1>
- <https://arxiv.org/pdf/2405.01229>
- <https://www.arxiv.org/pdf/2601.16466>
- <https://arxiv.org/pdf/2601.05742>
- <https://arxiv.org/html/2405.13077v2>
- <https://arxiv.org/html/2510.18728v1>
- <https://arxiv.org/pdf/2411.14502?>
- <https://arxiv.org/html/2502.09638v2>
- <https://arxiv.org/html/2404.01833v3>
- <https://arxiv.org/html/2402.13148>
- <https://arxiv.org/html/2402.13148v2>
- <https://arxiv.org/html/2404.01833v1>
- <https://arxiv.org/html/2405.13077>
- <https://openreview.net/pdf/c77928221487623b9e844434fc7061df1962b0de.pdf>
- <https://arxiv.org/html/2505.15738v2>
- <https://arxiv.org/html/2501.18638v2>
- <https://github.com/SahilHaiHum/llm-prompt-attacks-extended>
- <https://github.com/topics/jailbreak-prompts>
- <https://openreview.net/pdf?id=qmcbA7VPTd>
- <https://arxiv.org/html/2602.03265v1>
- <https://arxiv.org/html/2509.06350v1>
- <https://github.com/topics/ai-jailbreak>

- <https://arxiv.org/pdf/2410.15645>
- <https://github.com/NY1024/BAP-Jailbreak-Vision-Language-Models-via-Bi-Modal-Adversarial-Prompt>
- <https://github.com/WhileBug/AwesomeLLMJailBreakPapers>
- <https://github.com/SCAlabUnical/LLM-Bias-Jailbreak>
- <https://www.arxiv.org/pdf/2512.23173>
- <https://github.com/topics/ai-jailbreak-prompts>
- <https://arxiv.org/pdf/2509.06350>
- <https://openreview.net/forum?id=6yCZErufu9>
- <https://github.com/qizhangli/Adversarial-Prompt-Translator>
- <https://github.com/topics/jailbreak-prompt>
- <https://arxiv.org/html/2509.06350v2>
- <https://arxiv.org/html/2601.05742>
- <https://arxiv.org/html/2410.15645v2>
- <https://arxiv.org/html/2405.09113v2>
- <https://arxiv.org/html/2506.21972v1>
- <https://openreview.net/forum?id=hXA8wqRdyV>
- <https://arxiv.org/html/2601.05742v1>
- <https://arxiv.org/html/2501.18638>
- <https://openreview.net/pdf?id=ZuZujQ9LJV>
- <https://arxiv.org/html/2501.18638v1>
- <https://arxiv.org/html/2511.02376v3>
- <https://arxiv.org/html/2509.06350>
- <https://arxiv.org/html/2506.01307v1>
- https://github.com/CyberAlbSecOP/AwesomeGPTSuper_Prompting
- <https://arxiv.org/html/2404.01833v2>
- <https://arxiv.org/pdf/2307.08715v1>
- <https://arxiv.org/html/2410.11317v1>
- <http://arxiv.org/html/2510.18728v1>
- <https://github.com/alexmalan/ChatGPT-Waifu-Prompt-DAN-Jailbreak>
- https://github.com/cyberalbsecop/awesomegptsuper_prompting
- <https://github.com/topics/jailbreak>